國立臺灣大學電機資訊學院資訊網路與多媒體研究所

碩士論文

Graduate Institute of Networking and Multimedia

College of Electrical Engineering and Computer Science

National Taiwan University

Master’s Thesis

用於臺灣逐時容許對流天氣預報之少步數 MeanFlow 殘差生成

Few-Step MeanFlow Residuals for Hourly Convection-Allowing Forecasting over Taiwan

鍾鎮嶸

Desmond Cheong Zheng Yong

指導教授: 周承復 博士

Advisor: Cheng-Fu Chou, Ph.D.

中華民國 115 年 _____ 月

__________ 2026

doi:10.6342/NTU202601285

國立臺灣大學碩士學位論文

口試委員會審定書

用於臺灣逐時容許對流天氣預報之少步數
MeanFlow 殘差生成

Few-Step MeanFlow Residuals for
Hourly Convection-Allowing Forecasting over
Taiwan

本論文係鍾鎮嶸君（R13944064）在國立臺灣大學資訊網路與多媒體研究所完成之碩士學位論文，於民國 115 年 ______ 月 ______ 日承下列考試委員審查通過及口試及格，特此證明

口試委員：________________________________

（指導教授）

________________ ________________

________________ ________________

________________ ________________

________________ ________________

所　　長：________________________________

doi:10.6342/NTU202601285

doi:10.6342/NTU202601285

Acknowledgements

首先，我想感謝周承復教授這一年半以來的耐心指導，以及對我的信任。在 CMLab N 組進行的研究與計畫非常有趣，也伴隨著挑戰性。老師總是能夠為我們爭取許多算力資源；每當研究或計畫卡關、毫無頭緒的時候，周老師總能很快想到對的解決方案與方向。非常幸運能夠在臺大網媒所就讀碩士班時，遇到這麼好的指導教授。

我也很感謝在實驗室一起合作、努力、實作的學長姐與同學們。你們在各種知識與技能上的無私分享，讓我受益良多，也讓這段研究的日子更加充實而難忘。

再來要感謝爸媽、外公外婆、姐姐、弟弟，以及一路相伴的朋友們。謝謝你們始終的支持、包容與陪伴，在我求學的路上做我最溫暖、最堅強的後盾，讓我能夠無後顧之憂地專心完成學業。

最後，我要感謝 NVIDIA 開源 StormCast 與 PhysicsNeMo 的預訓練模型與訓練框架，以及中央氣象署與國家災害防救科技中心（NCDR）所提供的 RWRF 再分析與雷達降水資料；沒有這些開源資源與寶貴資料，本研究將難以完成。謹以此論文，獻給所有曾經幫助過我、陪我走過這段旅程的人。

doi:10.6342/NTU202601285

doi:10.6342/NTU202601285

摘要

近年來，基於擴散模型的生成式天氣預報（例如 GenCast 與 StormCast）已展現出在機率性、銳利化、且可集成的短時預報任務上超越傳統確定性深度學習模型的能力。然而，擴散模型在推論階段需要沿著機率流常微分方程（PF-ODE）執行數十次神經網路前向傳遞（Number of Function Evaluations, NFE），使其在進行長時間自迴歸滾動預測或大規模集成預報時的計算成本過高；同時，擴散殘差在稀疏、重尾的降水通道上仍偏向平滑，難以重現對流尺度的極端降水結構。

本論文以一個以臺灣區域（RWRF）資料集訓練的兩階段 StormCast 模型（凍結的確定性回歸均值加上生成式殘差頭）為基礎，提出並評估 **MeanFlow**（平均速度流匹配）作為其生成式殘差頭：網路直接學習一段完整時間區間上的平均速度 u_θ，使得單次前向傳遞即可橫越整段噪聲到資料的軌跡，推論僅需 1–2 次網路評估、完全不需數值積分。MeanFlow 與一般擴散蒸餾不同，無需教師、僅需單一訓練階段：它透過 MeanFlow 恆等式（以單次 Jacobian–向量積自舉、並停止梯度的目標）直接從資料學習平均速度場，當區間長度趨於零時其目標恰好退化為標準流匹配。我們將此目標移植到 StormCast 的標準化殘差管線上，並保留兩個對照基準：原生的 EDM 擴散殘差頭（需 35 NFE，即我們所欲加速的對象），以及以條件流匹配（Conditional Flow Matching, I-CFM）學習瞬時直線速度場 v_θ、需以少數幾步 Euler 積分取樣的 **FlowCast** 殘差頭——MeanFlow 在 75% 的批次上恰好退化為 FlowCast 的目標，故 MeanFlow 對 FlowCast 構成僅針對

doi:10.6342/NTU202601285

訓練目標本身的對照。針對臺灣 RWRF 目標僅有 4 個高解析度輸出通道（t2m, u10, v10, qpepre）的特性，我們設計逐通道加權的 L2 損失，並對稀疏重尾的降水通道 qpepre 加重權重；我們另外比較了 log1p 降水編碼與原始 mm/h 編碼，以及降水通道權重 w_{pr} 的取捨。

我們在 2022 年整年逐小時驗證資料上，以確定性誤差（RMSE/MAE）、機率分數（CRPS）、降水類別分數（CSI、FSS、HSS、誤報率），以及涵蓋 StormCast 主力 1–6 小時預報窗的自迴歸滾動誤差等指標，在相同訓練樣本預算（約 200 萬樣本）下將 MeanFlow 與 EDM 擴散基準及 FlowCast 進行正面比較。實驗結果顯示，MeanFlow 僅以 1–2 次網路評估（2 NFE，相較於擴散取樣器的 35 NFE；約為擴散取樣器十分之一、FlowCast 約三分之一的成本，可在單張 NVIDIA RTX A6000 上將一次 50 成員、24 小時的集成預報自約 32 分鐘縮短至約 4 分鐘，$K = 2$ 加速 7.9×、$K = 1$ 加速 9.9×）即可達到與擴散基準同級的確定性與類別技能，並取得全體最佳的降水類別命中率（CSI ／ HSS，於主力鄰域窗）以及全體各頭中最佳的溫度精度（t2m RMSE 較擴散基準低約 9%）。在 1–6 小時的自迴歸滾動中，MeanFlow（$K = 2$）於最前面數個預報時距取得最低的緯向風（u10）與溫度誤差（溫度於整段預報窗皆低於擴散基準），而 FlowCast 則於第一小時之後在經向風（v10）上領先。其主要代價為集成離散度偏窄，在自迴歸滾動下表現為小幅的 CRPS 落差（於僅 1 次評估時最為明顯，第 2 次評估可補回大部分離散度，故以 $K = 2$ 為建議運行點）。本研究為區域對流尺度生成式預報提供了一套可重現的單階段平均速度流匹配流程與降水導向的評估基準。

關鍵字：平均速度流匹配、MeanFlow、流匹配、擴散模型、單步生成、生成式天氣預報、對流尺度預報、降水臨近預報、StormCast

doi:10.6342/NTU202601285

Abstract

Diffusion-based generative weather forecasters such as GenCast and StormCast have recently surpassed deterministic deep-learning baselines on probabilistic, sharp, and ensemble-capable short-range prediction tasks. However, sampling a diffusion model requires integrating its probability-flow ODE with tens of neural-network function evaluations (NFE) per forecast step, which makes autoregressive rollouts and large ensembles computationally prohibitive for operational regional, convective-scale deployment; moreover, the diffusion residual remains comparatively smooth on the sparse, heavy-tailed precipitation channel, where convective-scale extremes matter most.

This thesis builds on a two-stage StormCast model trained on a Taiwan-domain RWRF dataset — a frozen deterministic regression mean plus a generative residual head — and proposes and evaluates **MeanFlow**, an average-velocity flow-matching objective, as the StormCast generative residual head. The network learns the mean velocity over an entire time interval, so a single forward pass transports the state across the whole noise-to-data

doi:10.6342/NTU202601285

trajectory and sampling needs only one to two network evaluations, with no numerical integration at all. Unlike diffusion distillation, MeanFlow is teacher-free and trained in a single stage: it learns the average-velocity field directly from data through the MeanFlow identity (a single-Jacobian–vector-product, stop-gradient bootstrapped target), and as the interval length shrinks to zero its objective collapses exactly to standard flow matching. We transcribe the objective into StormCast's standardised-residual pipeline and retain two reference heads for comparison: the native EDM diffusion residual (35 NFE — the model we set out to accelerate), and **FlowCast**, an Independent Conditional Flow Matching (I-CFM) head that learns the instantaneous straight-line velocity field v_θ and samples it with a handful of explicit Euler steps. Because MeanFlow's objective reduces exactly to FlowCast's on 75% of each batch, the MeanFlow-versus-FlowCast comparison is a controlled A/B test on the training objective alone. Because the Taiwan RWRF target contains only four high-resolution output channels (`t2m`, `u10`, `v10`, `qpepre`), we can afford channel-specific loss design: a channel-weighted L2 objective that up-weights the sparse, heavy-tailed precipitation channel `qpepre`. We additionally study the `log1p` versus raw-mm/h precipitation encoding and the precipitation channel-weight w_{pr}.

We compare MeanFlow head-to-head against the EDM diffusion baseline and the FlowCast head at a matched training-sample budget ($\approx$ 2 M samples) on the full 2022

doi:10.6342/NTU202601285

hourly validation year, using a precipitation-aware metric suite: per-channel RMSE/MAE, probabilistic CRPS, categorical precipitation scores (CSI, FSS, HSS, false-alarm ratio) at operational thresholds, and autoregressive rollout error across StormCast's headline 1–6 h forecast window. Our experiments show that MeanFlow reaches the EDM diffusion baseline's deterministic and categorical skill class at one to two network evaluations (2 NFE versus the diffusion sampler's 35) — roughly a tenth of the diffusion sampler's cost and about a third of FlowCast's, cutting a 50-member 24-hour ensemble forecast from ≈ 32 to ≈ 4 minutes on a single NVIDIA RTX A6000 ($7.9\times$ at $K = 2$, $9.9\times$ at $K = 1$) — while posting the best categorical precipitation hit rates (CSI/HSS) of any head at the headline neighbourhood and the best temperature accuracy of any head (`t2m` RMSE ≈ 9 % below the diffusion baseline). Across the 1–6 h rollout, MeanFlow ($K = 2$) holds the lowest zonal-wind (`u10`) and temperature error at the leading steps — temperature staying below the diffusion baseline across the whole horizon — while FlowCast leads the meridional wind (`v10`) beyond the first hour. Its principal cost is narrowed ensemble spread, which surfaces as a modest rollout-CRPS gap (largest when exactly one evaluation is used; a second evaluation restores most of the spread, so $K = 2$ is the recommended operating point). The thesis contributes a reproducible single-run average-velocity flow-matching pipeline for the StormCast residual and a precipitation-aware evaluation protocol

doi:10.6342/NTU202601285

for regional convective-scale generative forecasting.

Keywords: MeanFlow, Average-Velocity Flow Matching, Flow Matching, Diffusion Models, One-Step Generation, Generative Weather Forecasting, Convective-Scale Forecasting, Precipitation Nowcasting, StormCast

doi:10.6342/NTU202601285

Contents

doi:10.6342/NTU202601285

doi:10.6342/NTU202601285

doi:10.6342/NTU202601285

doi:10.6342/NTU202601285

List of Figures

doi:10.6342/NTU202601285

doi:10.6342/NTU202601285

List of Tables

doi:10.6342/NTU202601285

doi:10.6342/NTU202601285

Denotation

NWP	Numerical Weather Prediction
RWRF	Regional Weather Research and Forecasting (Taiwan domain reanalysis)
EDM	Elucidated Diffusion Model (Karras et al., 2022); the reference diffusion baseline
CFM	Conditional Flow Matching
I-CFM	Independent Conditional Flow Matching (the FlowCast baseline objective)
MeanFlow	Average-velocity flow matching (Geng et al., 2025); the method of this thesis
JVP	Jacobian–vector product (forward-mode, used for the MeanFlow target)
ODE	Ordinary Differential Equation
PF-ODE	Probability-Flow Ordinary Differential Equation

doi:10.6342/NTU202601285

SDE	Stochastic Differential Equation
NFE	Number of Function Evaluations (neural-net forward passes per sample)
EMA	Exponential Moving Average
CSI	Critical Success Index
FSS	Fractions Skill Score
HSS	Heidke Skill Score
FAR	False-Alarm Ratio
CRPS	Continuous Ranked Probability Score
M_t	High-resolution atmospheric state at time t (target). Channels: `t2m`, `u10`, `v10`, `qpepre`.
S_t	Low-resolution conditioning state at time t (24 channels).
I	Time-invariant fields (land–sea mask, orography).
F_ξ	Deterministic regression network producing mean $\mu_t = F_\xi(M_{t-1}, S_t, I)$.
μ_t	Regression mean prediction.
R_t	Residual $R_t = M_t - \mu_t$ learned by the generative head.
c	Conditioning bundle for the generative head. All three heads (EDM diffusion, FlowCast, MeanFlow) use concat(M_{t-1}, μ_t, I); the original StormCast additionally concatenates the low-resolution back-

doi:10.6342/NTU202601285

ground S_t.

D_ψ EDM diffusion baseline denoiser (EDMPrecond SongUNet).

v_θ FlowCast (I-CFM baseline) instantaneous velocity field (FlowCast-Precond SongUNet).

u_θ MeanFlow average-velocity field $u_\theta(z, r, t, c)$ (MeanFlowPrecond SongUNet); the network of this thesis.

σ EDM noise level.

$\sigma_{\text{min}}, \sigma_{\text{max}}$ Minimum and maximum EDM noise levels (0.002 and 80).

σ_{data} Data standard-deviation hyperparameter shared by all three heads (0.5).

ρ Karras schedule exponent (7).

t (or r, t) Flow time, in $[0, 1]$ (noise at 0, data at 1). MeanFlow uses an interval $[r, t]$ with state time $r \leq t$.

x_0, x_1 Flow endpoints: prior sample $x_0 \sim \mathcal{N}(0, I)$ and standardised residual $x_1 = R_t/\sigma_{\text{data}}$.

v Instantaneous (I-CFM) velocity field; conditional target $v = x_1 - x_0$.

$u(z_r, r, t)$ Average velocity of the flow over $[r, t]$; the field MeanFlow learns. $\lim_{t \to r} u = v$.

ρ_{MF} MeanFlow ratio: fraction of each batch with $r < t$ (the bootstrapped branch); 0.25.

doi:10.6342/NTU202601285

σ_{path}	I-CFM conditional-path standard deviation (0.01; MeanFlow uses the pure interpolant, $\sigma_{\text{path}} = 0$).
K	Number of sampling network evaluations (NFE): Euler steps for FlowCast, average-velocity segments for MeanFlow.
β_k	Per-channel loss weight; $w_{\text{pr}} = \beta_{\texttt{qpepre}}$ is the precipitation weight.

doi:10.6342/NTU202601285

Chapter 1 Introduction

1.1 Weather Forecasting and Its Societal Stakes

Accurate short-range weather forecasting is a public-safety service. In Taiwan, a small, densely populated island lying along the western Pacific typhoon track, the stakes are especially concentrated: a single landfalling typhoon can dump more than $1000\,$mm of rainfall on steep orography in twelve hours, trigger flash floods and landslides, and disrupt transportation, agriculture, and power. Between typhoons, Mei-yu frontal systems, afternoon convective storms, and orographically forced rainfall drive the remainder of the hydrological cycle. For decision-makers who must issue evacuation orders, schedule reservoir releases, or re-route air traffic, the relevant question is almost always probabilistic and local: how much rain, where, within the next six to twelve hours?

The operationally critical quantity — precipitation at convective scales — is also the hardest for weather models to predict. Precipitation fields are sparse (most pixels are dry), heavy-tailed (the few wet pixels span orders of magnitude), and strongly coupled to terrain and mesoscale dynamics that only emerge at kilometre-scale resolution. This combination means that any forecasting system which produces visually smooth outputs, or which regresses toward the mean, will systematically underpredict the very events that matter most.

doi:10.6342/NTU202601285

1.2 A Brief History of Weather Prediction Methods

Modern weather forecasting has passed through four broad eras, surveyed in detail in Chapter 2; we sketch them here only far enough to place this thesis.

For most of a century, operational forecasting has meant numerical weather prediction (NWP): numerically integrating the primitive equations of atmospheric motion on a three-dimensional grid, with unresolved sub-grid processes (convection, microphysics, boundary-layer turbulence, radiation) closed by physical parameterisations and uncertainty quantified by perturbed-physics ensembles. NWP is physically consistent and remarkably skilful, but convection-permitting ensembles are computationally expensive and operationally latency-bound. A long-standing complement is statistical post-processing — from model output statistics to gradient-boosted trees — which learns a local correction to the NWP forecast and established the precedent that learned components can usefully sit alongside a physical core.

Since the early 2020s a third era of purely data-driven global models (FourCastNet, Pangu-Weather, GraphCast) has rivalled or surpassed operational NWP at synoptic scales for a fraction of the inference cost. These deterministic models share one disqualifying weakness for convective forecasting: trained with pixelwise regression losses, they produce blurry fields that underestimate extremes. The generative turn resolves this by learning the distribution of plausible forecasts rather than its conditional mean — DGMR for radar nowcasting, GenCast at global scale, and StormCast at regional convection-permitting scale — and it is this generative line that the present thesis builds on and sets out to make fast.

doi:10.6342/NTU202601285

1.3 Diffusion Models for Weather: Intuition

A diffusion model is a generative model trained to reverse a gradual noising process. Given a clean sample x_0 drawn from some data distribution — say, a high-resolution precipitation field over Taiwan — we progressively add Gaussian noise until after enough steps the result is indistinguishable from pure noise. A neural network is then trained to undo one step of this process: given a noised sample and the current noise level, predict the underlying clean signal (equivalently, the score of the noised distribution). At sampling time we start from pure Gaussian noise and apply the learned denoiser repeatedly until we recover a clean sample.

Three properties make this framework attractive for forecasting — it is naturally probabilistic (different noise seeds give an ensemble for free), its samples stay sharp because the model is never asked to average over outcomes, and the denoiser can be conditioned on auxiliary inputs (the previous atmospheric state, coarse boundary fields, invariant land-surface information) — and GenCast at global scale and StormCast at regional scale both exploit them.

1.4 The Inference-Cost Problem

The cost of this flexibility is paid at inference time. High-quality diffusion sampling requires integrating the model's probability-flow ordinary differential equation (PF-ODE) with a few tens of neural-network forward passes per generated sample — the so-called number of function evaluations (NFE); the 18-step Heun schedule used by the baseline in this thesis costs 35 NFE per forecast step. For a single forecast step this is tolerable,

doi:10.6342/NTU202601285

but operational use cases compound it. An autoregressive 24-hour rollout multiplies the cost by 24. A 50-member ensemble for probabilistic forecasting multiplies it again. A convection-permitting Taiwan-domain forecast with a 50-member ensemble over a 24-hour horizon therefore requires on the order of $50 \times 24 \times 35 = 42{,}000$ forward passes of a large U-Net — which places the entire class of generative forecasters near the edge of, or outside, the operational envelope of a regional forecasting centre that must refresh its nowcast every few minutes between radar volumes.

The cost is not the only issue: the reason diffusion needs so many steps is that its PF-ODE trajectory is curved, and a curved path cannot be integrated accurately in one or two steps. The discretisation error is worst precisely on the sparse, heavy-tailed precipitation channel, where an aggressively sub-sampled diffusion residual reverts toward a smooth "slightly wet everywhere" field that scores acceptably on RMSE but destroys the convective structure operational users care about.

1.5 From Diffusion to Flow Matching

Both problems — the step count and the curvature — point at the same lever: the shape of the generative trajectory. If the path from noise to data were a straight line rather than a curved diffusion ODE, then a handful of explicit Euler steps would integrate it almost exactly, and a single step would already be a strong approximation.

Conditional Flow Matching (CFM) [14, 31] is a recently-proposed generative framework that learns exactly such a model. Instead of learning a score and integrating a curved PF-ODE, CFM regresses a deterministic velocity field that transports a Gaussian prior to the data along a straight-line probability path, and it does so with a simulation-free mean-

doi:10.6342/NTU202601285

squared-error objective that needs no teacher model at all. Ribeiro and Pucer [22] applied this idea to radar precipitation nowcasting under the name **FlowCast** and reported that a 1-to-10-step flow-matching sampler matched or beat a 100-step diffusion sampler on probabilistic precipitation skill at a fraction of the inference cost. A straight-line flow is integrated accurately in a handful of explicit Euler steps — but it is still integrated, and that residual handful of network evaluations is the last cost a flow-matching forecaster leaves on the table.

This thesis pushes the same lever one decisive notch further with **MeanFlow** [7]. Rather than learning the instantaneous velocity of the flow and then integrating it, MeanFlow learns the flow's average velocity over a whole time interval, so that one network evaluation transports a noise sample across the entire trajectory — no ODE solver, no discretisation error, and sampling collapses to one or two network evaluations. We adapt MeanFlow to the regional convection-permitting StormCast setting on the Taiwan RWRF domain as the generative residual head, keeping the two-stage decomposition (a frozen regression mean plus a generative residual head) intact and replacing only the residual head's curved EDM diffusion process. FlowCast's straight-line I-CFM velocity field is retained as the natural flow-matching baseline MeanFlow generalises — MeanFlow's objective reduces exactly to FlowCast's at zero interval length, so a comparison between them is a controlled A/B on the training objective — and the EDM diffusion residual as the reference baseline we set out to accelerate, all heads measured at a matched training-sample budget with data, architecture, and regression mean held fixed. Crucially, MeanFlow is not a distillation of the diffusion teacher: trained from scratch in a single run, it is a clean test of whether a better-shaped generative target alone — with no teacher signal — already delivers the speed and sharpness operational regional forecasting needs.

doi:10.6342/NTU202601285

1.6 Contributions

This thesis makes the following contributions:

1. We adapt the average-velocity **MeanFlow** objective [7] to the StormCast two-stage regression-plus-residual architecture as the generative residual head — to our knowledge its first application to a regional, convection-permitting, hourly forecaster (the Taiwan RWRF domain): the network learns the flow's displacement over whole time intervals via the MeanFlow identity (a single-JVP, stop-gradient bootstrapped target), so the residual is sampled in 1–2 network evaluations with no ODE solver, teacher-free and in a single training run.

2. We set up a controlled comparison by anchoring two reference heads to exactly the same data, frozen regression mean F_ξ, normalisation, σ_{data}, and backbone, so only the generative trajectory varies: the native EDM diffusion residual (the 35-NFE model we set out to accelerate) and **FlowCast**, the straight-line Independent-CFM velocity baseline [14, 22, 31] that MeanFlow generalises (the A/B of §1.5).

3. We exploit the four-channel target with a precipitation-aware channel-weighted L2 loss that up-weights the sparse, heavy-tailed `qpepre` channel, and ablate the precipitation channel weight w_{pr} and the `log1p`-versus-raw-mm/h encoding.

4. We assemble a precipitation-aware evaluation suite — per-channel RMSE/MAE, probabilistic CRPS, categorical skill scores (CSI, FSS, HSS, false-alarm ratio) at operational thresholds, and 1–6-hour rollout errors — and run the head-to-head comparison (plus an NFE/wall-clock-versus-quality trade-off and a legacy-baseline reference) on the full 2022 hourly validation year of the Taiwan RWRF dataset.

doi:10.6342/NTU202601285

5. We provide, in Appendix C, line-by-line derivations of the machinery MeanFlow rests on — the score–denoiser (Tweedie) identity, the VE-SDE / PF-ODE, Conditional Flow Matching, and the MeanFlow average-velocity identity — self-contained enough to reconstruct from scratch.

1.7 Thesis Outline

Chapter 2 surveys weather-prediction methods, a condensed account of the diffusion theory underlying the EDM baseline, and conditional flow matching building up to the average-velocity MeanFlow objective at the centre of this thesis, in enough depth to motivate and define every component used later. Chapter 3 describes the dataset, the frozen regression mean and the EDM diffusion and FlowCast baselines, and — as its core — the MeanFlow average-velocity residual pipeline as implemented in the accompanying codebase. Chapter 4 presents the experimental protocol and the head-to-head results across NFE budgets, precipitation thresholds, and rollout horizons. Chapter 5 summarises findings, limitations, and future work.

doi:10.6342/NTU202601285

doi:10.6342/NTU202601285

Chapter 2 Background and Related Work

This chapter assembles the theoretical background needed to understand the remainder of the thesis. It is organised in four parts: a survey of weather-prediction methods from NWP to data-driven models (Sections 2.1–2.4); a condensed account of score-based diffusion — the DDPM construction in summary and the elucidated EDM formulation used by our diffusion baseline (Sections 2.5–2.7), given only to the depth needed to understand the baseline we accelerate and the shared data scale σ_{data}; the theory of Conditional Flow Matching building up to the average-velocity **MeanFlow** objective at the centre of this thesis — with FlowCast's straight-line velocity field as the flow-matching baseline MeanFlow generalises — together with a direct comparison against the curved diffusion path and a survey of prior generative precipitation models (Sections 2.8–2.10). The precipitation-aware metrics used throughout (CSI, FSS, HSS, CRPS, rollout error) are defined where they are first used, in Section 4.2. Throughout the chapter we state the key identities in the main text and defer the derivations the method rests on to Appendix C.

doi:10.6342/NTU202601285

2.1 Numerical Weather Prediction

Numerical weather prediction discretises the primitive equations of atmospheric motion — continuity, momentum (Navier–Stokes on a rotating sphere), thermodynamic energy, and water-vapour conservation — onto a three-dimensional grid and advances the state forward in time via explicit or semi-implicit integration. Sub-grid processes not captured by the dynamical core (convection, microphysics, boundary-layer turbulence, radiation, land-surface exchange) are closed through physical parameterisations. Operational systems include ECMWF's IFS (global spectral, $0.1°$ horizontal resolution), NCEP's GFS, and regional limited-area models such as WRF and its Taiwan-tuned variant RWRF, which provide the high-resolution targets used in this thesis.

Uncertainty is quantified by ensemble prediction systems that perturb initial conditions (e.g. via singular-vector or ensemble Kalman methods) and model physics. A convection-permitting (sub-4 km) ensemble with a few tens of members and a 24-hour forecast horizon requires hours of dedicated supercomputer time. The practical consequence is that NWP ensemble size and spatial resolution are traded off against operational latency.

2.2 Data-Driven Global Weather Models

Starting in 2022, a series of deep-learning models demonstrated that a single neural network, trained on the ERA5 reanalysis, can produce global medium-range forecasts that rival operational NWP on many headline metrics at a fraction of the inference cost.

doi:10.6342/NTU202601285

FourCastNet. Pathak et al. [18] used an Adaptive Fourier Neural Operator to produce six-hourly global forecasts at $0.25°$ resolution. It was the first model to demonstrate that a data-driven system could match IFS on synoptic variables within a plausible compute budget.

Pangu-Weather. Bi et al. [2] introduced a hierarchical three-dimensional transformer with a hierarchical time-aggregation scheme and demonstrated medium-range skill competitive with or exceeding IFS on several variables and lead times.

GraphCast. Lam et al. [13] adopted a multi-mesh graph neural network defined on an icosahedral discretisation of the sphere. It produces ten-day $0.25°$ forecasts that are competitive with IFS on the majority of evaluated fields and does so in roughly one minute on a single TPU, compared with hours on a supercomputer for NWP.

These models share an important limitation: because they are trained with deterministic pixelwise losses (typically latitude-weighted MSE), their forecasts become progressively blurrier with lead time and systematically underestimate extreme events. For convection and precipitation, which this thesis cares about most, this is disqualifying.

2.3 Regional and Convective-Scale Forecasting

Global models operate at resolutions too coarse to resolve convection and orographic rainfall over Taiwan. Three lines of work fill the gap.

Nowcasting models. MetNet-1/2/3 [1, 4, 26] produce high-resolution short-range precipitation forecasts by ingesting ground-based radar and satellite observations through

doi:10.6342/NTU202601285

large convolutional/attention backbones. DGMR [21] uses a conditional GAN to produce sharp, ensemble-like radar nowcasts and was the first work to make a strong empirical case that generative modelling is essential for precipitation skill.

StormCast. Pathak et al. [19] introduced a convection-permitting regional diffusion forecaster trained on the High-Resolution Rapid Refresh (HRRR) analysis over CONUS. StormCast decomposes forecasting into two stages:

1. A deterministic regression network F_ξ (a `StormCastUNet`) takes the previous high-resolution state M_{t-1}, a low-resolution conditioning state S_t, and time-invariant fields I, and predicts the conditional mean $\mu_t = F_\xi(M_{t-1}, S_t, I)$.

2. A residual diffusion network D_ψ (an EDMPrecond SongUNet) models the residual $R_t = M_t - \mu_t$ conditionally on $c = \text{concat}(M_{t-1}, S_t, \mu_t, I)$.

At inference time the model is rolled out autoregressively, with the previous diffusion output feeding the next step's regression input. This decomposition has two payoffs: the deterministic network captures what is easy to predict with a single network call, leaving the diffusion model free to focus on the residual uncertainty; and the residual is closer to Gaussian than the raw fields, which stabilises diffusion training.

This thesis uses a Taiwan-adapted StormCast model trained on the RWRF dataset. The adaptation reduces the output state from the 99 channels of the CONUS original to just four: two-metre temperature (`t2m`), ten-metre zonal and meridional winds (`u10`, `v10`), and hourly precipitation (`qpepre`). This channel reduction has a useful side-effect: with only four output channels, per-channel loss engineering becomes cheap, and we exploit this in the precipitation-aware losses of Chapter 3.

doi:10.6342/NTU202601285

2.4 GenCast and the Generative Turn

Price et al. [20] introduced GenCast, a global 0.25° conditional diffusion forecaster that samples ensemble members by varying the initial noise seed. GenCast was the first diffusion forecaster to clearly surpass the leading deterministic model (GraphCast) across the majority of its evaluation suite, on both deterministic scores (RMSE) and probabilistic scores (CRPS). Its success is the strongest empirical argument for the claim that motivates this thesis: generative modelling — and diffusion in particular — is the right inductive bias for short-to-medium range weather forecasting, but its operational cost is the obstacle to practical deployment.

2.5 Score-Based Diffusion: A Primer

A diffusion model defines a forward stochastic process that progressively corrupts a data sample $x_0 \sim p_{\text{data}}$ with Gaussian noise until the terminal state is indistinguishable from pure noise, then learns a neural network that reverses the corruption. The apparent difficulty of this programme — fitting the full generative reverse process of a high-dimensional distribution — hides a sequence of algebraic identities that together reduce training to a mean-squared-error regression. Because diffusion enters this thesis only as the reference baseline we set out to accelerate (and as the lineage of the curved generative path that flow matching straightens), we give it a condensed treatment: the discrete-time DDPM construction of Ho et al. [11] is summarised below, and the elucidated EDM parameterisation [12] that the baseline actually uses — and from which all three heads inherit the data scale σ_{data} — follows in Section 2.6. The continuous-time identities the baseline

doi:10.6342/NTU202601285

relies on — the variance-exploding SDE and its probability-flow ODE [29], Tweedie's score–denoiser formula, and EDM preconditioning — are derived in Appendix C.

The denoising diffusion probabilistic model (DDPM) of Ho et al. [11] defines a T-step Markov chain that adds Gaussian noise to a clean sample under a closed-form marginal, and after the standard variational-bound algebra and an ϵ-reparameterisation its training objective collapses — with no loss-bearing approximation — to a single mean-squared-error denoising regression: an intractable maximum-likelihood problem reduces to supervised denoising, with one network call per training step. The cost lands at inference, where the ancestral sampler draws $T \approx 1000$ stochastic reverse steps, far too slow for deployment. The continuous-time EDM parameterisation the baseline actually uses follows in Section 2.6; the discrete-time DDPM/DDIM machinery is used by no head in this thesis and is not developed further.

2.6 EDM: The Elucidated Formulation

Karras et al. [12] proposed a unified parameterisation of diffusion training and sampling that cleans up a number of ad-hoc choices in earlier DDPM-style models. The EDM diffusion baseline of this thesis uses this formulation in full, and FlowCast borrows its data scale σ_{data} from it (Chapter 3). The EDM preconditioning scalings are not heuristics: they are the unique one-parameter family that makes the denoiser's input statistics, target statistics, and per-σ loss scale simultaneously uniform across σ [12].

doi:10.6342/NTU202601285

Noise schedule. EDM uses a ρ-spaced discretisation of noise levels between σ_{min} and σ_{max}:

$$\sigma_i = \left(\sigma_{\text{max}}^{1/\rho} + \frac{i}{N-1}\left(\sigma_{\text{min}}^{1/\rho} - \sigma_{\text{max}}^{1/\rho}\right)\right)^{\rho}, \tag{2.1}$$

for $i = 0, \ldots, N-1$, with $\sigma_{\text{min}} = 0.002$, $\sigma_{\text{max}} = 80$, and $\rho = 7$. The schedule concentrates noise levels near σ_{min}, which is where most of the useful signal is generated.

Preconditioning. A naive denoiser $F_\theta(x_\sigma, \sigma)$ must cope with inputs whose magnitudes vary by orders of magnitude. EDM addresses this by wrapping the inner network in skip, input, output, and noise scalings:

$$D_\theta(x_\sigma; \sigma) = c_{\text{skip}}(\sigma)\, x_\sigma + c_{\text{out}}(\sigma)\, F_\theta\big(c_{\text{in}}(\sigma)\, x_\sigma,\ c_{\text{noise}}(\sigma)\big), \tag{2.2}$$

where

$$c_{\text{skip}}(\sigma) = \sigma_{\text{data}}^2/(\sigma^2 + \sigma_{\text{data}}^2), \qquad c_{\text{out}}(\sigma) = \sigma \cdot \sigma_{\text{data}}/\sqrt{\sigma^2 + \sigma_{\text{data}}^2},$$
$$c_{\text{in}}(\sigma) = 1/\sqrt{\sigma^2 + \sigma_{\text{data}}^2}, \qquad c_{\text{noise}}(\sigma) = \tfrac{1}{4}\log\sigma.$$

These scalings ensure that F_θ always sees inputs of unit variance and predicts an unnormalised residual, which is numerically stable across the full range of σ.

Training loss. EDM trains with a weighted denoising-score-matching loss:

$$\mathcal{L}_{\text{EDM}} = \mathbb{E}_{x_0,\sigma,\epsilon}\Big[\lambda(\sigma)\, \|D_\theta(x_0 + \sigma\epsilon; \sigma) - x_0\|^2\Big], \tag{2.3}$$

with a log-normal distribution over σ and a Karras-specified weighting $\lambda(\sigma)$.

doi:10.6342/NTU202601285

Heun sampler. EDM samples via a second-order Heun integrator of the PF-ODE. Given x_i at noise level σ_i, the next sample is

$$d_i = (x_i - D_\theta(x_i; \sigma_i))/\sigma_i, \tag{2.4}$$

$$x_{i+1}^{(\text{euler})} = x_i + (\sigma_{i+1} - \sigma_i)\, d_i, \tag{2.5}$$

$$d_i' = (x_{i+1}^{(\text{euler})} - D_\theta(x_{i+1}^{(\text{euler})}; \sigma_{i+1}))/\sigma_{i+1}, \tag{2.6}$$

$$x_{i+1} = x_i + (\sigma_{i+1} - \sigma_i)\, \tfrac{1}{2}(d_i + d_i'). \tag{2.7}$$

Each step uses two denoiser evaluations except the final step where the correction is skipped. With $N = 18$ steps this yields $2N - 1 = 35$ NFE, matching the original StormCast paper, which samples its residual diffusion with the same EDM second-order solver for 18 denoising steps [19]. The baseline in this thesis keeps this native $N = 18$ schedule, and its 35-NFE Heun sampler is the cost figure that the straight-line FlowCast sampler of Chapter 3 is benchmarked against: every NFE that FlowCast saves at matched forecast quality is a direct operational speedup.

2.7 Conditioning in Diffusion Models

Conditional diffusion models can be implemented either by concatenating the condition c along the channel axis of the denoiser's input, or by classifier-free guidance (CFG) [10], which trains the network jointly on conditional and unconditional tasks and linearly combines their outputs at inference time. StormCast uses the simpler concatenation approach: the conditioning bundle is stacked with the noised target along the channel dimension and fed to the SongUNet. No classifier-free guidance is used. The original StormCast concatenates $c = \text{concat}(M_{t-1}, S_t, \mu_t, I)$; all three generative heads in this thesis — the

doi:10.6342/NTU202601285

EDM diffusion baseline, FlowCast, and MeanFlow — instead condition on the trimmed bundle concat(M_{t-1}, μ_t, I), dropping the low-resolution background S_t identically (Section 3.5). Because the trim is applied to every head alike, the fundamental change between the diffusion baseline and the flow-matching heads remains the generative process itself, not the way conditioning enters the network.

2.8 Flow Matching, FlowCast, and MeanFlow

Conditional Flow Matching (CFM) is a recently-proposed alternative to score-based diffusion that learns the same kind of noise-to-data generative model — a deterministic flow integrated from a simple prior to the data distribution — but with a training objective that bypasses score matching entirely. CFM is the foundation on which the method of this thesis is built: the average-velocity **MeanFlow** objective (Section 3.6) generalises the straight-line CFM velocity field of the **FlowCast** baseline (Section 3.5). We therefore develop CFM in full — the simulation-free training principle, the straight-line I-CFM path of FlowCast, and finally the average-velocity identity of MeanFlow — giving it more depth than the condensed diffusion primer above.

Continuous normalising flows, simulation-free training. A continuous normalising flow (CNF; 3) parameterises a deterministic velocity field $v_\theta(x, t)$, $t \in [0, 1]$, and pushes a prior sample $x_0 \sim \mathcal{N}(0, I)$ along the ODE $\dot{x}_t = v_\theta(x_t, t)$ to obtain x_1, which we would like to distribute as p_{data}. Classical maximum-likelihood CNF training requires evaluating $\log p_\theta(x_1)$ via the Jacobian trace over the whole trajectory — a prohibitive cost at image or video scale. Lipman et al. [14] and Tong et al. [31] instead train v_θ by direct regression against a target vector field $u_t(x \mid z)$ associated with a tractable conditional probability

doi:10.6342/NTU202601285

path $p_t(x \mid z)$ indexed by a latent z:

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t\sim\mathcal{U}(0,1),\, z\sim q(z),\, x\sim p_t(x|z)}\Big[\|v_\theta(x,t) - u_t(x \mid z)\|^2\Big]. \tag{2.8}$$

The key theoretical result [14, Thm. 2] is that the gradient of (2.8) with respect to θ equals the gradient of the marginal flow-matching loss with the intractable marginal target field — so training the network against a conditional target yields an unbiased estimator of the true field. A derivation of this identity, along with the constants that are dropped because they do not depend on θ, is given in Appendix C, §C.3.

Independent CFM (I-CFM) and the straight-line path. The simplest instantiation, Independent CFM [31], takes $z = (x_0, x_1)$ with $x_0 \sim \mathcal{N}(0, I)$ independent of $x_1 \sim p_{\text{data}}$, and defines the straight-line Gaussian probability path

$$p_t(x \mid x_0, x_1) = \mathcal{N}\big((1-t)\,x_0 + t\,x_1,\ \sigma_{\text{path}}^2 I\big), \qquad x_t = (1-t)\,x_0 + t\,x_1 + \sigma_{\text{path}}\,\eta,\ \ \eta \sim \mathcal{N}(0, I). \tag{2.9}$$

Differentiating the conditional mean with respect to t gives the corresponding target vector field

$$u_t(x \mid x_0, x_1) = x_1 - x_0, \tag{2.10}$$

which is independent of x — exactly the straight-line constant-velocity interpolation between the noise sample and the data sample. The training objective becomes

$$\mathcal{L}_{\text{I-CFM}} = \mathbb{E}_{t,x_0,x_1,\eta}\Big[\big\|v_\theta\big((1-t)x_0 + tx_1 + \sigma_{\text{path}}\eta,\ t\big) - (x_1 - x_0)\big\|^2\Big]. \tag{2.11}$$

The path noise $\sigma_{\text{path}} > 0$ "thickens" the otherwise-singular conditional paths and regularises the vector-field fit, following Tong et al. [31]; FlowCast uses $\sigma_{\text{path}} = 0.01$. A

doi:10.6342/NTU202601285

derivation of (2.10) from (2.9) and an explicit comparison with the curved probability flow of DDPM are in Appendix C, §C.4.

FlowCast. Ribeiro and Pucer [22] introduced FlowCast, the first fully probabilistic application of I-CFM to precipitation nowcasting: a single U-Net backbone (an Earthformer-style cuboid-attention network [5] in a VAE latent space) regresses the straight-line vector field $v_\theta(x_t, t, c)$ conditioned on a past-observation bundle c, sampled with a 10-step Euler integrator. On SEVIR [32], FlowCast at 1 ODE step beat DDIM [27] at 100 steps on CRPS and CSI and matched or exceeded CasCast [8] and PreDiff [6] at a fraction of the cost. Two features make it the natural flow-matching baseline here: the straight-line ODE is by construction the trajectory any one-step sampler approximates, so the 1-step-to-10-step gap is much smaller than for curved diffusion; and training is simulation-free and teacher-free, learning the field directly from data in a single run. Section 3.5 adapts it to the StormCast residual pipeline as the baseline MeanFlow generalises.

MeanFlow: average velocity for one-step generation. Even a straight-line flow is sampled by integrating an ODE, and that integration is what keeps the budget at $K \approx 10$: the marginal field the network learns is curved (the average over crossing conditional paths), so each Euler step pays a truncation error. A separate line of work removes the integral from the sampler altogether. Consistency models [28, 30] map any point of a trajectory directly to its endpoint, but impose this as a constraint on the network's behaviour that needs carefully tuned discretisation curricula and trains unstably. MeanFlow [7] reaches the same goal field-theoretically: it defines the average velocity $u(z, r, t)$ — the displacement of the flow over $[r, t]$ divided by the interval length (Figure 2.1) — as a ground-truth field induced by v, and derives an exact identity relating the two ($u = v - (t - r)\,\frac{d}{dt}u$

doi:10.6342/NTU202601285

in the paper's data-at-t=0 convention). Training regresses the network onto this identity using only the same conditional-velocity samples as CFM, the derivative term a single stop-gradient Jacobian–vector product, so no integral, teacher, or curriculum is needed; at $r = t$ it collapses to standard flow matching. One evaluation then transports a prior sample across the whole flow — 1–2 NFE by construction. On ImageNet 256×256 a from-scratch MeanFlow reaches FID 3.43 at 1 NFE, a 50–70% relative improvement over the best previous one-step models [7]. Section 3.6 transcribes the identity into this thesis's noise-to-data convention and adapts MeanFlow as the residual head, under hyperparameters identical to FlowCast.

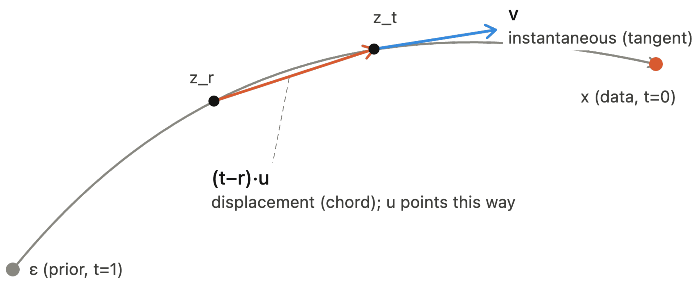


Figure 2.1: Instantaneous versus average velocity along a flow trajectory — the geometric distinction between flow matching (the FlowCast baseline) and MeanFlow. The grey curve is the marginal flow path from the prior ε to a data sample x. At a point z_t the instantaneous velocity v (blue) is the tangent to the path: it is the field a flow-matching model learns, and sampling must integrate it step by step. The average velocity u over the interval $[r, t]$ points along the chord from z_r to z_t (orange), since by definition $z_t - z_r = (t - r)\,u$ — so a single evaluation of u jumps across the whole interval with no integration, which is the quantity MeanFlow learns. As $t \to r$ the chord collapses onto the tangent and $u \to v$, recovering flow matching as the zero-interval limit. The schematic follows the original MeanFlow paper's data-at-t=0 (prior-at-t=1) convention; Section 3.6 mirrors it to this thesis's noise-at-0 convention, but the chord-versus-tangent geometry is identical either way.

doi:10.6342/NTU202601285

2.9 Flow Matching versus Diffusion: Why Straight-Line Paths Help

Both score-based diffusion and Conditional Flow Matching learn a deterministic ODE that carries a Gaussian prior sample to a data sample; the difference is the shape of the path, and that shape governs how few steps a sampler can use. The EDM probability-flow ODE $\mathrm{d}x/\mathrm{d}\sigma = -\sigma\,\nabla_x \log p_\sigma(x)$ (§C.2) has a velocity set by the score of the noised marginal, which changes nonlinearly with σ, so its trajectory is curved and a low-order integrator accumulates truncation error as the step grows — which is why high-quality diffusion sampling sits in the 20–50-NFE band and even fast solvers such as DPM-Solver [16] bottom out near ten evaluations, the error surfacing first on the sparse precipitation channel where the score is most ill-conditioned at small σ. Independent CFM (Section 2.8) instead uses the straight-line interpolation $x_t = (1-t)x_0 + tx_1$ with constant target velocity $u_t = x_1 - x_0$: along each conditional path the curvature is zero, so an Euler step is exact on the conditional path and the only residual error comes from the learned marginal field averaging over crossing paths. The gap between 1-step and 10-step flow-matching sampling is therefore far smaller than between 1-step and 25-step diffusion — the claim Ribeiro and Pucer [22] verified on radar nowcasting and that Chapter 4 re-tests on Taiwan RWRF. The difference is also organisational: making diffusion cheap normally means distillation (progressive [24], consistency [28, 30], adversarial [25]), which needs a converged teacher, a second stage, and careful sampler/preconditioning matching, whereas reshaping the path itself — rectified flow [15], FlowCast, and MeanFlow [7] (which goes further and removes the integral entirely) — trains the velocity field directly from data in a single teacher-free run. This thesis accordingly compares a single-run flow forecaster against the diffusion

doi:10.6342/NTU202601285

model it would otherwise have to distil, not distillation schemes against one another.

2.10 Prior Generative Models for Precipitation Nowcasting

Generative precipitation modelling has converged on the same conclusion from several directions: DGMR [21] made the case with a conditional GAN, GenCast [20] and StormCast [19] with score-based diffusion residuals at global and regional scale, and NVIDIA's CorrDiff [17] with an EDM super-resolution residual — all sharing the multi-step diffusion sampler as their common cost — while Ribeiro and Pucer [22] introduced FlowCast as the first fully-probabilistic I-CFM nowcaster, matching or beating diffusion nowcasters (CasCast [8], PreDiff [6]) at a fraction of their NFE on SEVIR [32]. To the best of our knowledge, the present thesis is the first to adapt conditional flow matching — and in particular the average-velocity MeanFlow objective [7] — to a regional convection-permitting forecaster (as opposed to a radar-to-radar nowcaster) anchored to a frozen NWP-style regression mean in the StormCast two-stage decomposition, and to report a three-way diffusion-versus-CFM-versus-MeanFlow comparison with precipitation-specific metrics on the Taiwan domain.

doi:10.6342/NTU202601285

Chapter 3 Method

This chapter describes what was actually built. Section 3.1 states the residual-forecasting problem in precise terms. Section 3.2 describes the Taiwan RWRF dataset and its normalisation. Section 3.3 documents the underlying data sources and the preprocessing pipeline that turns them into the training Zarr store. Section 3.4 recaps the frozen regression mean and the EDM diffusion residual that serves as our reference baseline. Section 3.5 describes the FlowCast Conditional Flow Matching residual — the straight-line flow-matching baseline that the method of this thesis generalises — which we develop first because MeanFlow is defined relative to it. Section 3.6, the core of the chapter, describes **MeanFlow**, the average-velocity flow-matching residual that is the method of this thesis: it pushes the sampling budget down to one or two network evaluations with no ODE integration, while keeping every other design choice identical to the FlowCast baseline. Section 3.7 describes the dataset-specific loss modifications for the precipitation channel, which are shared by the diffusion baseline and both flow-matching heads.

doi:10.6342/NTU202601285

3.1 Problem Setup

We adopt the StormCast two-stage decomposition throughout. A frozen deterministic regression network F_ξ produces a conditional mean

$$\mu_t = F_\xi(M_{t-1}, S_t, I), \tag{3.1}$$

and a generative residual head models the residual $R_t = M_t - \mu_t$ conditioned on a bundle assembled from the previous state, the regression mean, and the invariants. The regression mean is the same frozen checkpoint for every method in this thesis, so the only object that varies is the generative residual head.

We compare three interchangeable residual heads, holding the dataset, the regression mean μ_t, the backbone architecture, and the data scale σ_{data} fixed:

- **EDM diffusion (reference baseline).** A frozen-recipe EDMPrecond SongUNet D_ψ models the residual as a score-based diffusion and samples it by integrating the PF-ODE from $\sigma_{\max}$ to $\sigma_{\min}$ with a Heun sampler ($N = 18$ steps, $2N - 1 = 35$ NFE per hourly step). This is the model we set out to accelerate, and the reference against which the flow-matching heads are measured; it is the residual head of the original StormCast, save for the trimmed conditioning bundle that it shares with them (Section 3.5).
- **FlowCast (flow-matching baseline).** A FlowCastPrecond SongUNet velocity field v_θ models the residual as a straight-line Conditional Flow Matching ODE and samples it with K explicit Euler steps (K NFE per hourly step), where we sweep K from 1 to 50 (the head-to-head scoreboard reports $K \in \{10, 15, 20\}$). It is the straight-

doi:10.6342/NTU202601285

line special case that MeanFlow generalises, retained as the natural flow-matching comparator.

- **MeanFlow (the method of this thesis).** A MeanFlowPrecond SongUNet u_θ models the average velocity of the same flow over an interval $[r, t]$ [7], so that one network evaluation transports the state across the whole interval with no ODE integration. Sampling uses K average-velocity segments at exactly K NFE per hourly step, with $K \in \{1, 2\}$ as the headline operating points (Section 3.6). Training hyperparameters are kept identical to the FlowCast baseline, and MeanFlow's objective reduces exactly to FlowCast's at zero interval length, making MeanFlow-versus-FlowCast an A/B test on the training objective alone.

In all cases the full prediction at each step is $\hat{M}_t = \mu_t + \hat{R}_t$, and the model is rolled out autoregressively, with the previous generated state feeding the next step's regression input. The central empirical question of the thesis is whether the single-run, teacher-free MeanFlow head matches or beats the diffusion head at a substantially smaller NFE (and wall-clock) budget — $K \in \{1, 2\}$ network evaluations against the diffusion sampler's 35, with the FlowCast baseline ($K \approx 10$) marking the straight-line flow-matching reference point in between — all at a matched training-sample budget so that the comparison reflects the generative trajectory rather than training compute. Neither flow head distils D_ψ or queries it during training; the three heads are trained independently on the same data. Figure 3.1 shows one autoregressive step of the resulting pipeline.

doi:10.6342/NTU202601285

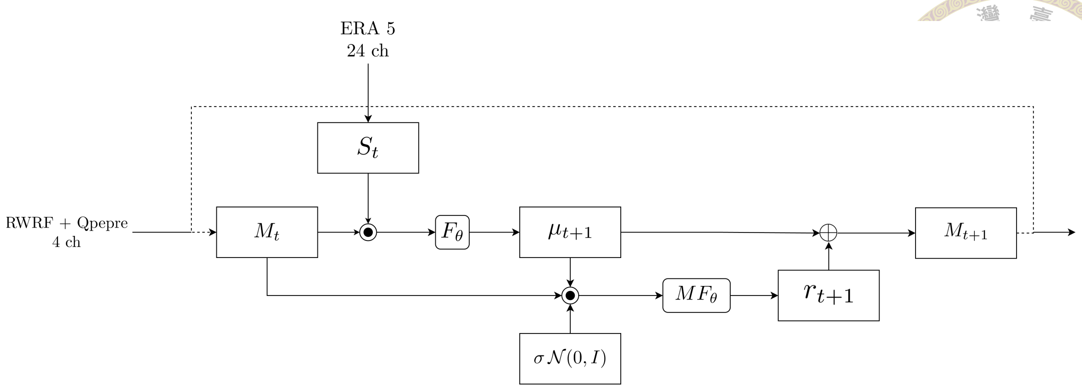


Figure 3.1: One autoregressive step of the two-stage StormCast–MeanFlow forecaster (the method of this thesis). The previous high-resolution state M_t (RWRF + QPEPRE, four channels) and the low-resolution ERA5 conditioning S_t (24 channels) feed the frozen regression network F_θ (the deterministic F_ξ of the text), which predicts the conditional mean μ_{t+1}. The MeanFlow residual head MF_θ (the average-velocity network u_θ), conditioned on the regression mean and driven by a Gaussian draw $\sigma\mathcal{N}(0,I)$, produces the residual r_{t+1} in one or two network evaluations with no ODE integration; the next state is $M_{t+1} = \mu_{t+1} + r_{t+1}$ and is fed back as the input to the following step (dashed lines). The schematic indexes the produced fields at $t+1$; the main text writes the same step as $\mu_t = F_\xi(M_{t-1}, S_t, I)$ and $\hat{M}_t = \mu_t + \hat{R}_t$.

3.2 Dataset: Taiwan RWRF

The training data live in a Zarr store under the project directory (see Appendix B). The store has three top-level groups.

LowRes conditioning S_t. Shape $(T, 24, 224, 128)$, where T is the hourly time index and the 24 channels are low-resolution re-gridded ERA5 fields serving as synoptic-scale boundary conditioning: four surface variables plus humidity, temperature, winds, and geopotential at four pressure levels (1000 hPa, 850 hPa, 500 hPa and 250 hPa). The exact variable list and provenance are given in Section 3.3.

HighRes target M_t. Shape $(T, 4, 224, 128)$, with four channels: two-metre temperature (`t2m`), ten-metre zonal and meridional winds (`u10`, `v10`), and hourly accumulated precipitation (`qpepre`). The horizontal grid covers Taiwan from $21.6°$ to $25.6°$N and $119.75°$ to

doi:10.6342/NTU202601285

122.25°E at approximately 2 km resolution. The dataset splits into 21,216 training hours (2019-08 through 2021-12) and 8,760 validation hours (calendar year 2022).

Invariants I. Shape $(2, 224, 128)$, channels land–sea mask and orography. These are broadcast across time and concatenated into the conditioning bundle.

Normalisation. Per-channel means and standard deviations are computed on the training split and stored in the Zarr metadata. The HighRes statistics are

$$\mu_{\mathrm{HR}} = (295.98,\ -1.58,\ -2.65,\ 0.182), \qquad \sigma_{\mathrm{HR}} = (5.61,\ 3.84,\ 5.66,\ 9.12), \tag{3.2}$$

which show that `qpepre` has a near-zero mean with standard deviation an order of magnitude larger than its mean — a direct fingerprint of its heavy-tailed, sparse distribution. Training and validation losses are computed on normalised fields; metrics in Chapter 4 are reported on denormalised fields with physical units.

Cleaned variant used for the reported experiments. The shapes above describe the original store. All FlowCast-versus-diffusion runs in this thesis are trained and evaluated on a cleaned re-export of the same data, in which the precipitation channel is encoded as $\log(1+x)$ before standardisation (the `qpepre_log1p` option of the loader) and the spatial domain is centre-cropped to a U-Net-friendly 192×96 grid. The cleaned store covers 18,765 training hours (2019-08-01 through 2021-12-31) and 8,759 validation hours (calendar year 2022). The asymmetric drop from the original counts is expected: the joint-validity mask removes 1,803 missing-QPEPRE hours and 648 lone-survivor pairs (whose $+1$ h neighbour is invalid) from the training span, whereas every one of those gaps falls

doi:10.6342/NTU202601285

in 2019–2021, so the 2022 validation year is essentially complete (only its final unpaired hour is dropped). The $\log(1 + x)$ encoding is the single most important preprocessing choice for the precipitation channel: it compresses `qpepre`'s dynamic range from a raw standard deviation of $9.12\,\mathrm{mm\,h^{-1}}$ to roughly 0.37 in log space, which makes the standardised residual far better-conditioned for every residual head (Section 4.7). Because the spatial crop changes the field of view, RMSE/CRPS magnitudes are not directly comparable to the legacy 224×128 baseline of Section 4.9; we flag this wherever the two are tabulated together. The regression mean and all three residual heads are trained on this same cleaned, log1p-encoded store, which is what the "train on the same qpepre encoding" gotcha in Section 4.11 refers to.

3.3 Data Sources and Preprocessing Pipeline

Section 3.2 described the Zarr store consumed by the training loop. This section describes where the underlying fields come from and how the preprocessing pipeline in `data_preprocessing/` assembles them. The design follows a low-resolution guidance, high-resolution learning strategy: a coarse global reanalysis provides synoptic-scale conditioning, and a convection-permitting regional model plus a radar-derived precipitation product provides the fine-scale target.

3.3.1 Raw data sources

ERA5 (low-resolution input, S_t). The ECMWF ERA5 reanalysis [9] at approximately $25\,\mathrm{km}$ spatial resolution supplies the synoptic-scale environmental state. From ERA5 we take four surface variables (mean sea-level pressure `mslp`, two-metre temperature `t2m`, and

doi:10.6342/NTU202601285

ten-metre winds `u10`, `v10`) and five upper-air variables (specific humidity `q`, temperature `t`, zonal wind `u`, meridional wind `v`, and geopotential height `z`) at four pressure levels $\{1000, 850, 500, 250\}$ hPa, for a total of $4 + 5 \times 4 = 24$ channels — the LowRes bundle S_t of Section 3.2.

RWRF (high-resolution target, part of M_t). The Radar-assimilated Weather Research and Forecasting (RWRF) numerical model provides convection-permitting ($\approx$2 km) analyses over Taiwan. We use hourly RWRF output (`wrfout_d01_*_interp`) for the three near-surface dynamical channels: `t2m`, `u10`, `v10`. RWRF contains the fine-scale dynamic and thermodynamic structures that the complex Taiwan terrain imprints on near-surface weather, which is exactly what a regional convective-scale forecaster must learn.

QPEPRE (high-resolution target, precipitation channel). Quantitative Precipitation Estimation (QPEPRE) is a radar-based hourly-rainfall product distributed as fixed-grid text files (`qpepre_YYYYMMDDHHMM-YYYYMMDDHHMM_1_h.txt`), one per hour. Unlike the RWRF fields, which are model outputs, QPEPRE is an observational product and therefore serves as a more faithful ground truth for surface rainfall than any model-forecast precipitation field. It occupies the fourth HighRes channel, `qpepre`.

Invariants I. Two static fields are extracted once from an RWRF file and broadcast across time: the land–sea mask `lsm` and orography `orog`. These let the network condition on terrain without re-reading static fields at every time step.

Table 3.1 summarises the three sources.

doi:10.6342/NTU202601285

Table 3.1: Raw data sources used for training. Low-resolution ERA5 provides synoptic conditioning; RWRF and QPEPRE together supply the convection-permitting target.

	ERA5	RWRF	QPEPRE
Type	Reanalysis	Numerical model	Radar observation
Resolution	≈25 km	≈2 km	≈2 km
Role	LowRes input S_t	HighRes target M_t	HighRes target M_t
Channels used	24 (4 sfc + 5 × 4 levels)	3 (`t2m`, `u10`, `v10`)	1 (`qpepre`)
Native format	NetCDF	NetCDF (`wrfout`)	ASCII text

3.3.2 Preprocessing pipeline

Two staged scripts under `data_preprocessing/` assemble the store: `nc_month_to_day/` splits monthly ERA5 NetCDF files into single-hour files, and `nc_to_zarr/` harmonises ERA5, RWRF, and QPEPRE onto a shared 224×128 Taiwan grid (lat $[21.6, 25.6]$°N, lon $[119.75, 122.25]$°E at ≈2 km) and writes a consolidated StormCast-style Zarr store with $T_{\text{train}} = 21{,}216$ training hours (2019-08-01 through 2021-12-31) and $T_{\text{valid}} = 8{,}760$ validation hours (calendar year 2022). The pipeline performs file indexing, joint-validity precomputation, per-source extraction and re-gridding, single-pass (Welford) streaming statistics, and consolidated Zarr writing; the scripts and the exact invocation are in the `data_preprocessing/` tree (Appendix B). Three of its properties are load-bearing for the work here:

- Pixel-aligned multi-source grids. All heads consume conditioning and target on the same grid, so residual losses are computed pixel-wise with no on-the-fly re-gridding.
- Explicit `valid` mask. Roughly 10–15 % of hours fail the joint-validity check (usually a missing QPEPRE file); both loaders and metrics respect the mask, so scores are never contaminated by NaN-filled cells.

doi:10.6342/NTU202601285

- Frozen training-split statistics. The store commits to a single pair $(\mu_{\mathrm{HR}}, \sigma_{\mathrm{HR}})$ at write time (Equation 3.2), so all residual heads see identical normalised inputs — a prerequisite for the σ_{data}-matching of Section 3.4.

3.3.3 Target Distributions: Sparsity, Gaussianity, and the Diffusion Prior

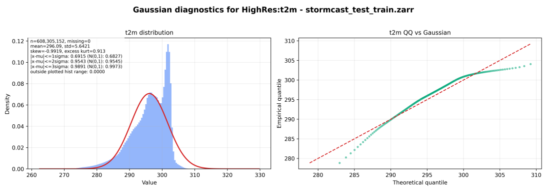
Diffusion models place a Gaussian prior on their target: the forward process is $x_\sigma = x_0 + \sigma\,\varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$, and the denoiser is trained under the implicit assumption that the marginals of x_0 are reasonably well-behaved relative to that noise scale. How closely each of the four HighRes channels actually conforms to this assumption directly shapes how the generative residual head must be regularised. Figure 3.2 shows per-channel histograms (with a fitted Gaussian overlay) and Gaussian QQ plots computed over the full training split ($n = 6.08 \times 10^8$ pixels per channel). The streaming statistics are summarised in Table 3.2. These diagnostics are computed on the original 224×128 raw-mm h^{-1} store (before the spatial crop and the `log1p` encoding of Section 3.2); the qualitative conclusion — three near-Gaussian channels and one pathological precipitation channel — carries over to the cleaned store, where the `log1p` transform is precisely the response to the `qpepre` non-Gaussianity diagnosed here.

Table 3.2: Summary statistics of the four HighRes target channels over the training split. A standard Gaussian has skew $= 0$, excess kurt $= 0$, and concentrates $68.3/95.5/99.7\,\%$ of its mass within $1/2/3\sigma$.

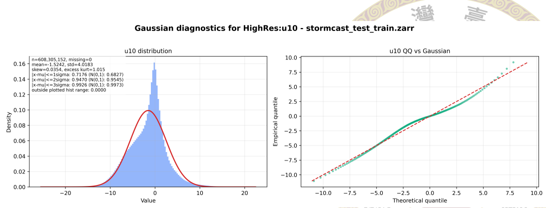
Channel	Mean	Std	Skewness	Excess kurt.	% in 1σ	% in 3σ
`t2m`	296.09	5.64	-0.99	0.91	69.2	98.9
`u10`	-1.52	4.02	0.04	1.01	71.8	99.3
`v10`	-2.32	5.50	-0.28	0.02	69.3	99.8
`qpepre`	0.18	6.86	-866.6	9.25×10^5	99.4	99.9

The three near-surface state channels are approximately Gaussian. `u10` is the closest:

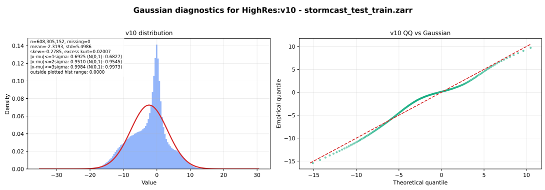
doi:10.6342/NTU202601285



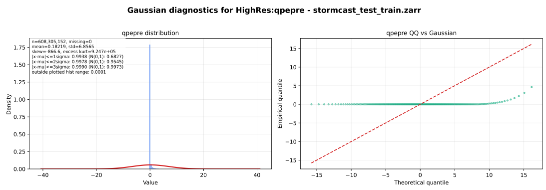
(a) t2m — 2 m temperature (K).



(b) u10 — 10 m zonal wind ($\mathrm{m\,s^{-1}}$).



(c) v10 — 10 m meridional wind ($\mathrm{m\,s^{-1}}$).



(d) qpepre — radar precipitation ($\mathrm{mm\,h^{-1}}$).

Figure 3.2: Per-channel distribution and Gaussian QQ plots for the four HighRes target channels over the 2.5-year training split. Left subpanels: empirical density (red) with a Gaussian fit of matched (μ, σ) (blue). Right subpanels: empirical quantiles against theoretical Gaussian quantiles; the red dashed line is $y = x$ for a perfect Gaussian. Generated by `data_preprocessing/inspect_tools/plot_high_res_vars.py`.

almost zero skewness, mild leptokurtosis, and a QQ plot that tracks the diagonal out to roughly $\pm 4\sigma$ before fanning into the heavy tails associated with gust events. v10 is almost symmetric with essentially zero excess kurtosis. t2m is the least symmetric of the three: left-skewed (skew $= -0.99$) with a secondary mode around 280 K corresponding to winter cold surges over northern Taiwan and the surrounding sea. For all three channels the empirical $1\sigma/2\sigma/3\sigma$ coverage agrees with the Gaussian reference to within a percentage point, which is exactly the regime for which the EDM noise schedule with $\sigma_{\text{data}} = 0.5$ was designed: the generative residual head sees each channel as a mild deformation of a unit-variance Gaussian after normalisation, and the standard ℓ_2 objective is a reasonable fit.

qpepre is a different object entirely. Its marginal is dominated by an atom at zero — the overwhelming majority of pixel-hours are dry — with a long, very sparse tail of heavy rainfall events. The raw statistics make this quantitative: the coverage inside $\pm 1\sigma$ is $99.4\,\%$ rather than the Gaussian $68.3\,\%$, which is the signature of a near-delta distri-

doi:10.6342/NTU202601285

bution whose standard deviation is inflated entirely by the tail. The empirical moments reported in Table 3.2 (skewness ≈ -867, excess kurtosis $\approx 10^6$) are formally enormous because the raw `qpepre` field uses a -9999 sentinel for missing observations that is only filtered at the next pipeline stage; for the same reason the diagnostic's `qpepre` standard deviation here (6.86) is computed by this offline inspection pass and need not equal the stored normalisation $\sigma_{\texttt{qpepre}} = 9.12$ of Equation 3.2, which the pipeline commits separately on the filtered training split — both are dominated by the tail, and only the stored value feeds normalisation. The physical tail above zero is still extreme (excursions beyond $100\,\mathrm{mm\,h^{-1}}$ occur during typhoon landfalls), but the numbers headline the practical message: the `qpepre` histogram looks nothing like a Gaussian, and the QQ plot sits on a flat shelf near zero that peels almost vertically at the upper end.

This mismatch drives three design decisions that recur throughout the thesis. First, score matching on raw `qpepre` is ill-posed at small σ (the score diverges as the target approaches a point mass at zero), which the $\log(1+x)$ encoding of Section 3.7 compresses and which the score-free straight-line FlowCast field sidesteps. Second, because dry pixels vastly outnumber rainy ones, a plain ℓ_2 loss converges to a "slightly wet everywhere" solution, motivating the `qpepre` up-weighting of Section 3.7 and the precipitation-specific metrics of Chapter 4. Third, with only one of four channels pathological — versus the 99 of global StormCast — per-channel loss weights β_k are a cheap, natural knob. In short, Figure 3.2 is the quantitative justification for every "qpepre is the hard channel" decision that follows: three channels live comfortably inside the Gaussian prior diffusion assumes, and one does not.

doi:10.6342/NTU202601285

3.4 Regression Mean and the EDM Diffusion Baseline

The reference model is the two-stage StormCast described in Section 2.3, with the EDM hyperparameters of Section 2.6. It supplies both the frozen regression mean that FlowCast reuses unchanged and the diffusion residual that FlowCast is compared against.

- **Regression F_ξ.** A `StormCastUNet` taking (M_{t-1}, S_t, I) as input and predicting μ_t. The checkpoint is used without modification or further training, and is shared by the diffusion baseline, FlowCast, and MeanFlow so that all three residual heads model a residual relative to the same mean.
- **Diffusion baseline D_ψ.** An EDMPrecond SongUNet trained to denoise the residual $R_t = M_t - \mu_t$ conditioned on $c_t = \mathrm{concat}(M_{t-1}, \mu_t, I)$ (the trimmed bundle shared by all three heads; Section 3.5). EDM hyperparameters: $\sigma_{\mathrm{min}} = 0.002$, $\sigma_{\mathrm{max}} = 80$, $\sigma_{\mathrm{data}} = 0.5$, $\rho = 7$; sampling is the Heun integrator of Section 2.6. The baseline is trained from scratch on the same cleaned dataset as FlowCast and evaluated at the same ≈ 2 M-training-sample budget (the nearest saved checkpoint; see Section 4.1).

Both networks are instantiated through the factory `get_preconditioned_architecture` in `stormcast/utils/nn.py`, which selects the EDM or FlowCast preconditioning wrapper by name. The conditioning bundle is assembled by `build_network_condition_and_target`, which handles the channel-wise concatenation and the one-hot time-offset encoding used in the rollout.

Why the baseline matters for a fair comparison. Beyond sharing the dataset, the frozen F_ξ, and the SongUNet backbone (`channel_mult` $= [1, 2, 2, 2, 2]$; Section 3.1),

doi:10.6342/NTU202601285

all three heads anchor to the same data scale $\sigma_{\text{data}} = 0.5$: both flow heads standardise their residual by exactly this value (Sections 3.5 and 3.6), so the per-channel loss scaling and regression-mean anchoring are numerically identical across heads. A mismatched σ_{data} would silently shift the residual distribution one head sees relative to the others and make any quality difference uninterpretable.

3.5 The FlowCast Baseline (Conditional Flow Matching)

FlowCast is the straight-line flow-matching baseline of this thesis, and we develop it first because the MeanFlow method of Section 3.6 is defined relative to it — MeanFlow reuses FlowCast's conditioning, standardised-residual pipeline, backbone, and losses unchanged, and its objective reduces exactly to FlowCast's at zero interval length. FlowCast replaces the diffusion baseline's curved PF-ODE with a simulation-free straight-line flow: a single network is trained from scratch to regress the target vector field $u_t = x_1 - x_0$ of Independent CFM (Section 2.8), and sampling is a handful of Euler steps. FlowCast is not a distillation method — it never queries the EDM diffusion model and uses no teacher at all — it is a one-training-run residual head that learns its velocity field directly from data. The same StormCast residual decomposition is preserved: the deterministic regression mean μ_t is produced by the frozen F_ξ, and FlowCast learns only the residual $R_t = M_t - \mu_t$ conditioned on a bundle c_t assembled by `build_network_condition_and_target` in `stormcast/utils/nn.py`.

Conditioning bundle. FlowCast and the EDM diffusion baseline are conditioned on exactly the same channel bundle. Both share the config setting

doi:10.6342/NTU202601285

```
regression_conditions: ["state", "background", "invariant"]
diffusion_conditions: ["state", "regression", "invariant"]
```

which is identical in `stormcast/config/model/flowcast.yaml` and `stormcast/config/model/diffusion.yaml`: the frozen regression network F_ξ is fed the full (M_{t-1}, S_t, I), but the generative network — diffusion <u>or</u> flow — is conditioned only on

$$c_t = \mathrm{concat}\big(M_{t-1},\ \mu_t,\ I\big). \tag{3.3}$$

The low-resolution background S_t is deliberately omitted from the <u>direct</u> conditioning path of every head (a deviation from the original StormCast, which concatenates S_t alongside μ_t); the synoptic-scale signal in S_t enters the residual only through the regression mean $\mu_t = F_\xi(M_{t-1}, S_t, I)$, which already summarises what the residual needs to know about it. Trimming S_t removes 24 input channels at no measured cost, and the trim is applied <u>identically</u> to all heads (the MeanFlow config carries the same two condition lists verbatim). Keeping `"regression"` in the diffusion-conditions list anchors every residual to the same μ_t, so the only thing that differs between the EDM, FlowCast, and MeanFlow residuals is the generative trajectory, not what they condition on — which is what makes the head-to-head comparison of Chapter 4 fair.

Standardised-residual pipeline. FlowCast operates on the residual in a <u>standardised</u> space. The loss (`stormcast/utils/flowcast_loss.py`) divides the raw residual by $\sigma_{\text{data}} = 0.5$ before building the flow trajectory, and the sampler (`stormcast/utils/nn.py:flowcast_model_forward`) multiplies the integrated state by σ_{data} on output. Keeping σ_{data} matched to the EDM baseline means the regression-mean anchoring statistics and the per-channel loss scaling are identical across the residual heads. Concretely,

doi:10.6342/NTU202601285

with $x_1 = R_t/\sigma_{\text{data}}$ and $x_0 \sim \mathcal{N}(0, I)$, the I-CFM interpolated state is

$$x_t = (1-t)\,x_0 + t\,x_1 + \sigma_{\text{path}}\,\eta, \qquad t \sim \mathcal{U}(t_\varepsilon, 1-t_\varepsilon),\ \eta \sim \mathcal{N}(0, I), \tag{3.4}$$

with $\sigma_{\text{path}} = 0.01$ from Tong et al. [31] and $t_\varepsilon = 10^{-5}$ clamping the endpoints. The small $\sigma_{\text{path}}\eta$ perturbation thickens the otherwise-Dirac conditional path $p_t(x_t \mid x_0, x_1)$ into a narrow Gaussian tube, separating it from the deterministic-interpolant ($\sigma_{\text{path}} = 0$) limit. The "Independent" in I-CFM refers instead to the coupling: x_0 and x_1 are drawn independently [31], as opposed to the minibatch optimal-transport coupling of OT-CFM.

Velocity predictor. The FlowCast network is the `FlowCastPrecond` module at `stormcast/utils/flowcast_precond.py`, instantiated through the factory call `get_preconditioned_architectur` …) at `stormcast/utils/nn.py`. The factory wraps a SongUNet with `channel_mult=[1, 2, 2, 2, 2]`, identical to the EDM baseline's backbone. Unlike `EDMPrecond` there are no $c_{\text{skip}}/c_{\text{in}}/c_{\text{out}}$ scalings — the network output is the velocity estimate:

$$v_\theta(x_t, t, c_t) = \text{SongUNet}\big(\text{concat}[x_t, c_t],\ t \cdot \tau\big), \tag{3.5}$$

where $\tau = 1000$ (the `time_scale` field in `stormcast/config/model/flowcast.yaml`) rescales the flow time $t \in [0, 1]$ before it enters the SongUNet's positional-embedding layer. The SongUNet positional embedding was designed for diffusion-timestep-valued inputs with dynamic range of order 10^2–10^3; feeding raw $t \in [0, 1]$ would collapse the embedding to a nearly-constant signal. $\tau = 1000$ restores enough dynamic range for the embedding to carry useful time information without changing the SongUNet itself, so no architectural changes from the diffusion baseline's backbone are needed.

doi:10.6342/NTU202601285

Algorithm 1 FlowCast training step (I-CFM on the standardised residual)

Require: frozen F_ξ, FlowCast velocity field v_θ, schedule $(\sigma_{\text{data}}, \sigma_{\text{path}}, t_\varepsilon)$, channel weights β

sample mini-batch $\{(M_{t-1}, S_t, M_t)\}$; form $\mu_t \leftarrow F_\xi(M_{t-1}, S_t, I)$, $R_t \leftarrow M_t - \mu_t$
build conditioning $c_t \leftarrow \text{concat}(M_{t-1}, \mu_t, I)$ ▷ note: S_t reaches v_θ only via μ_t
standardise: $x_1 \leftarrow R_t/\sigma_{\text{data}}$
sample $x_0 \sim \mathcal{N}(0, I)$, $t \sim \mathcal{U}(t_\varepsilon, 1 - t_\varepsilon)$, $\eta \sim \mathcal{N}(0, I)$
build interpolant: $x_t \leftarrow (1-t)\,x_0 + t\,x_1 + \sigma_{\text{path}}\,\eta$
target field: $u_t \leftarrow x_1 - x_0$
predict: $\hat{v} \leftarrow v_\theta(x_t, t, c_t)$
loss: $\mathcal{L} \leftarrow \|\hat{v} - u_t\|_{2,\beta}^2$
AdamW step on θ; EMA update $\theta^- \leftarrow 0.999\,\theta^- + 0.001\,\theta$

Loss. The training loss is the standardised-space I-CFM regression

$$\mathcal{L}_{\text{FC}} = \mathbb{E}\big[\,\|v_\theta(x_t, t, c_t) - (x_1 - x_0)\|_{2,\beta}^2\,\big], \tag{3.6}$$

where $\|\cdot\|_{2,\beta}$ is the per-channel-weighted L2 norm from Section 3.7 with $\beta = (1, 1, 1, 2)$ (the `channel_weights` field). Algorithm 1 summarises the per-step update.

Optimisation. Following the FlowCast paper, training uses AdamW (learning rate 5×10^{-4}, weight decay 10^{-4}) with a 4,000-step linear warmup into cosine decay, global gradient clipping at 1.0, and NaN-gradient sanitisation, for 400,000 steps at a global batch of 96 (the launcher overrides the config default of 64); the full hyperparameter list is in Appendix A. The batch size fixes the step-to-sample conversion of the matched-budget comparison (Section 4.1), anchored at $\approx 2\,$M training samples seen rather than a fixed optimiser-step count.

EMA shadow network. A second `FlowCastPrecond` is kept as a fixed-decay (0.999) exponential-moving-average shadow of the online weights (`stormcast/utils/ema.py`); a single fixed decay suffices because the straight-line velocity target is stationary, with

doi:10.6342/NTU202601285

no shrinking time discretisation to track. The shadow weights, saved to `ema_state.pt`, are used at validation and inference — loading the raw `checkpoint_*.pt` instead yields visibly noisier samples.

Sampling. Generation integrates the learned ODE $\dot{x}_t = v_\theta(x_t, t, c_t)$ from $t = 0$ to $t = 1$ with a fixed-step solver. The default in `flowcast_model_forward` (`stormcast/utils/nn.py`) is explicit Euler with K steps:

$$x_{t+\Delta t} = x_t + \Delta t\, v_\theta(x_t, t, c_t), \qquad \Delta t = 1/K, \qquad x_{t=0} \sim \mathcal{N}(0, I), \tag{3.7}$$

which costs K NFEs (a second-order `solver="midpoint"` variant at $2K$ NFEs is also implemented but unused in the reported experiments). The sampler finishes by de-standardising: $\hat{R}_t = \sigma_{\text{data}}\, x_{t=1}$ and $\hat{M}_t = \mu_t + \hat{R}_t$. Validation inside the training loop uses K = `valid_num_steps` $= 10$ Euler steps, while the main experiments report the scoreboard at $K \in \{10, 15, 20\}$ and sweep K from 1 to 50, placing each cost point against the EDM Heun baseline on the NFE/wall-clock–quality trade-off of Chapter 4.

Why a straight-line flow is the right residual head. Three properties of I-CFM make FlowCast more than a faster imitation of diffusion. The straight-line path is by construction what a one-step generator approximates, so its $K = 1/2$ numbers are a strong reference for the best a few-step sampler can do and the gap to $K = 10$ is small (Chapter 4), unlike the steep diffusion degradation below ≈ 20 NFE. Training is teacher-free and single-stage, avoiding the second run and the sampler/preconditioning/EMA-coupling subtleties a distillation would need. And Ribeiro and Pucer [22] report I-CFM is noticeably more stable than score matching on sparse heavy-tailed fields — directly relevant to

doi:10.6342/NTU202601285

qpepre, and re-tested here on the Taiwan domain.

3.6 The MeanFlow Residual (Average-Velocity Flow Matching)

FlowCast still pays for its samples with $K \approx 10$ Euler steps, because what it learns is the instantaneous velocity of a marginal field whose trajectories are curved (Section 2.8): every Euler step incurs truncation error against that curvature, and the error only vanishes as K grows. MeanFlow [7] removes the integration from the sampler instead of refining it. It trains the network to predict the average velocity of the flow over a whole interval $[r, t]$, so that a single evaluation transports the state across the interval exactly — no ODE solver, and therefore no discretisation error to amortise. We adapt MeanFlow to the StormCast residual pipeline as the generative residual head of this thesis: it reuses the conditioning bundle c_t, the standardised-residual convention, the SongUNet backbone, and the per-channel loss weights of the FlowCast baseline unchanged, and differs from it only in the training objective and the (correspondingly cheaper) sampler. MeanFlow is likewise teacher-free and trained from scratch in a single run.

Average velocity. Work in the standardised residual space of Section 3.5: $x_1 = R_t/\sigma_{\text{data}}$ is the data endpoint, $x_0 \sim \mathcal{N}(0, I)$ the prior, and the linear path $z_s = (1-s)\,x_0 + s\,x_1$ runs from noise at $s = 0$ to data at $s = 1$ with conditional velocity $v = x_1 - x_0$. (MeanFlow uses the pure interpolant, $\sigma_{\text{path}} = 0$: the identity below differentiates along the exact trajectory, so the Gaussian thickening used by FlowCast is dropped.) For two times $r \leq t$ the average

doi:10.6342/NTU202601285

velocity is the displacement over $[r, t]$ divided by the interval length,

$$u(z_r, r, t) \triangleq \frac{1}{t-r}\int_r^t v(z_\tau, \tau)\, d\tau, \qquad \lim_{t\to r} u(z_r, r, t) = v(z_r, r), \tag{3.8}$$

where $v(\cdot, \tau)$ is the marginal velocity field of Section 2.8. Like v, the field u is a property of the underlying flow, not of any network. Its defining property is that it makes transport across an interval a single algebraic step,

$$z_t = z_r + (t-r)\, u(z_r, r, t), \tag{3.9}$$

exactly, with no solver: if u is known, one evaluation at $(r, t) = (0, 1)$ maps noise to data. The cost has been moved from inference into training: Eq. (3.8) contains an intractable path integral, so u cannot be regressed directly.

The MeanFlow identity. The integral is eliminated by differentiation rather than quadrature: differentiating $(t-r)\, u(z_r, r, t) = \int_r^t v\, d\tau$ with respect to the state time r (holding t fixed, with z_r moving along the trajectory) and applying the product rule and the fundamental theorem of calculus gives the MeanFlow identity

$$\boxed{u(z_r, r, t) \;=\; v(z_r, r) \;+\; (t-r)\, \frac{d}{dr}\, u(z_r, r, t),} \tag{3.10}$$

whose total derivative expands along the trajectory tangent into partials,

$$\frac{d}{dr}\, u(z_r, r, t) \;=\; v(z_r, r)\, \partial_z u \;+\; \partial_r u. \tag{3.11}$$

The line-by-line derivation, including the sign flip relative to the data-at-0 convention of Geng et al. [7], is given in Appendix C, §C.6. The identity relates the field we want (u)

doi:10.6342/NTU202601285

to the field we can sample (v) using only quantities available at a single point of a single trajectory — the path integral is gone — and at $t = r$ it collapses to $u = v$, recovering plain flow matching as a boundary condition.

Training objective. A network $u_\theta(z_r, r, t, c_t)$ is encouraged to satisfy Eq. (3.10) by regressing onto the identity's right-hand side with the network's own derivative bootstrapped in:

$$\mathcal{L}_{\mathrm{MF}} = \mathbb{E}\Big[\,\mathrm{sg}(w)\,\big\|u_\theta(z_r, r, t, c_t) - \mathrm{sg}(u_{\mathrm{tgt}})\big\|_{2,\beta}^2\Big], \qquad u_{\mathrm{tgt}} = v + (t - r)\big(v\,\partial_z u_\theta + \partial_r u_\theta\big), \tag{3.12}$$

where $v = x_1 - x_0$ is the conditional I-CFM velocity standing in for the marginal field (the same substitution FlowCast makes, valid in expectation), sg is stop-gradient, and $\|\cdot\|_{2,\beta}$ is the channel-weighted norm of Section 3.7 with the same $\beta = (1, 1, 1, 2)$ as FlowCast. Three implementation choices follow Geng et al. [7]:

- **JVP bootstrap with stop-gradient.** The derivative in u_{tgt} is evaluated by a single forward-mode Jacobian–vector product through u_θ with tangent $(v, 1, 0)$ for (z, r, t) — one extra forward-mode pass, no higher-order gradients, because the whole target is stop-gradient (`torch.func.jvp` under `no_grad`). A network achieving zero loss satisfies the identity, and hence the definition Eq. (3.8).

- **Two-time sampling with partial collapse.** Each sample draws two i.i.d. times from $\mathcal{U}(t_\varepsilon, 1 - t_\varepsilon)$ and sorts them into $r \le t$. Only a fraction $\rho_{\mathrm{MF}} = 0.25$ of the batch keeps $r < t$ and trains on the bootstrapped target; the remaining 75% collapses $t \leftarrow r$, for which $u_{\mathrm{tgt}} = v$ and the objective reduces exactly to the FlowCast I-CFM regression. The collapsed subset pins the boundary condition $u(\cdot, r, r) = v$ that the

doi:10.6342/NTU202601285

Algorithm 2 MeanFlow training step (average-velocity matching on the standardised residual)

Require: frozen F_ξ, average-velocity network u_θ, scale σ_{data}, clamp t_ε, MeanFlow ratio ρ_{MF}, adaptive (p, ε_w), channel weights β

sample mini-batch $\{(M_{t-1}, S_t, M_t)\}$; form $\mu_t \leftarrow F_\xi(M_{t-1}, S_t, I)$, $R_t \leftarrow M_t - \mu_t$
build conditioning $c_t \leftarrow \text{concat}(M_{t-1}, \mu_t, I)$; standardise $x_1 \leftarrow R_t/\sigma_{\text{data}}$
sample $x_0 \sim \mathcal{N}(0, I)$; set $v \leftarrow x_1 - x_0$
sample (r, t): two i.i.d. $\mathcal{U}(t_\varepsilon, 1-t_\varepsilon)$ draws, sorted so $r \le t$; with prob. $1-\rho_{\text{MF}}$ collapse $t \leftarrow r$
interpolate state: $z_r \leftarrow (1-r)\,x_0 + r\,x_1$
if $r < t$: $\frac{d}{dr}u \leftarrow \text{jvp}\big(u_\theta(\cdot,\cdot,\cdot,c_t);\ (z_r, r, t);\ (v, 1, 0)\big)$ ▷ forward-mode, no grad
  $u_{\text{tgt}} \leftarrow v + (t-r)\,\frac{d}{dr}u$ **else**: $u_{\text{tgt}} \leftarrow v$
predict (single forward over the full batch): $\hat{u} \leftarrow u_\theta(z_r, r, t, c_t)$
per-sample error: $\Delta^2 \leftarrow \|\hat{u} - \text{sg}(u_{\text{tgt}})\|^2_{2,\beta}$; weight $w \leftarrow (\Delta^2 + \varepsilon_w)^{-p}$
loss: $\mathcal{L} \leftarrow \text{mean}\big(\text{sg}(w)\,\Delta^2\big)$
AdamW step on θ; EMA update $\theta^- \leftarrow 0.999\,\theta^- + 0.001\,\theta$

$r < t$ subset propagates outward; Geng et al. [7] find $\rho_{\text{MF}} = 0.25$ optimal, and the JVP overhead is confined to a quarter of the batch. The prediction pass itself runs once over the full batch, which keeps distributed training simple.

- **Adaptive weighting.** The per-sample weight $w = (\|\Delta\|^2_{2,\beta} + \varepsilon_w)^{-p}$ with $p = 1$, $\varepsilon_w = 10^{-3}$, applied with stop-gradient, implements the adapted-loss-weight scheme that Geng et al. [7] found necessary for stable bootstrapped training (their Tab. 1e); $p = 0$ recovers plain weighted MSE.

Algorithm 2 summarises the update.

Network parameterisation: two times into one backbone. The network is the `MeanFlowPrecond` module (`stormcast/utils/meanflow_precond.py`), wrapping the <u>same</u> SongUNet backbone (`channel_mult` $= [1, 2, 2, 2, 2]$) as the EDM and FlowCast heads. The only new requirement is injecting two times instead of one. The state time r takes FlowCast's route: it is scaled by $\tau = 1000$ and fed through the SongUNet's diffusion-time positional embedding (Eq. (3.5)). The interval length $\Delta = t - r$ enters through the SongUNet's otherwise-

doi:10.6342/NTU202601285

unused augmentation-label pathway as a fixed 32-dimensional sin / cos Fourier feature vector with log-spaced frequencies in $[1, 10^3]$:

$$u_\theta(z, r, t, c_t) = \text{SongUNet}\big(\text{concat}[z, c_t],\ r \cdot \tau,\ \text{FF}(t - r)\big). \tag{3.13}$$

This is the (r, Δ) variant of the two-time embeddings ablated by Geng et al. [7], who find time-plus-interval conditioning the most effective; the added parameters (one linear map from the 32 Fourier features) are negligible against the 78.7 M backbone. At $\Delta = 0$ the parameterisation reduces to FlowCast's up to a constant augmentation vector, which is what makes the per-parameter comparison between the two students clean.

Sampling. Generation replaces numerical integration with Eq. (3.9) applied over K equal segments of $[0, 1]$ (`meanflow_model_forward` in `stormcast/utils/nn.py`):

$$z_{(i+1)/K} = z_{i/K} + \tfrac{1}{K}\, u_\theta\big(z_{i/K},\ \tfrac{i}{K},\ \tfrac{i+1}{K},\ c_t\big), \qquad i = 0, \ldots, K-1, \qquad z_0 \sim \mathcal{N}(0, I), \tag{3.14}$$

at exactly one NFE per segment. $K = 1$ is the pure one-step sampler $\hat{x}_1 = x_0 + u_\theta(x_0, 0, 1, c_t)$; $K = 2$ trades one extra evaluation for a mid-trajectory correction and is the validation default. Unlike the FlowCast Euler sampler, increasing K here does not reduce a truncation error — each segment is already exact under a perfect u — it instead shortens the intervals over which the <u>learned</u> u_θ must be accurate. The sampler finishes identically to FlowCast: $\hat{R}_t = \sigma_{\text{data}}\, x_{t=1}$ and $\hat{M}_t = \mu_t + \hat{R}_t$.

Optimisation and checkpoints. Everything FlowCast-side is kept (AdamW, 5×10^{-4} learning rate, warmup-into-cosine schedule, gradient clipping, fixed-decay 0.999 EMA saved to `ema_state.pt`; Appendix A). The production run trains on 7 GPUs at a global

doi:10.6342/NTU202601285

batch of 112, placing the Chapter 4 checkpoint essentially on the $\approx 2\,$M-sample anchor shared by the EDM and FlowCast heads; the head-to-head harness loads the FlowCast and MeanFlow online checkpoints identically, so the A/B comparison is symmetric end to end.

Why average velocity for this residual. It attacks the only cost FlowCast leaves on the table — the straight-line path made each Euler step cheap, and MeanFlow removes the need to take many of them, compressing the residual head to 1–2 NFE per forecast hour. It is also the right shape for a controlled study (the $t = r$ subset of the objective is FlowCast's, so any skill difference in Chapter 4 is attributable to the average-velocity bootstrap alone) and avoids the unstable curriculum schedules of the principled alternative, consistency-style models: MeanFlow's target is a property of the underlying field, needs no curriculum, and trains with the same channel-weighted L2 machinery already validated on `qpepre`.

3.7 Loss Design for the Precipitation Channel

The four-channel RWRF target makes per-channel loss engineering cheap, and we use this budget to attack the hardest channel, `qpepre`, through a per-channel weight in the regression loss. The channel-weighted L2 term below is shared by the EDM baseline, FlowCast, and MeanFlow; the two flow heads apply it to their velocity regression.

doi:10.6342/NTU202601285

Channel-weighted L2. Let $\beta = (\beta_{\text{t2m}}, \beta_{\text{u10}}, \beta_{\text{v10}}, \beta_{\text{qpepre}})$ be a vector of non-negative per-channel weights. The weighted L2 norm used by the FlowCast velocity loss is

$$\|a\|_{2,\beta}^2 = \sum_{k=1}^{4} \beta_k \, \|a_k\|^2, \tag{3.15}$$

where the sum runs over the four output channels and the norm inside each term is the ordinary spatial L2. The default configuration uses $\beta = (1, 1, 1, w_{\text{pr}})$, with the precipitation weight w_{pr} exposed as the last entry of `channel_weights` in `stormcast/config/model/flowcast.yaml`. The headline FlowCast run uses $w_{\text{pr}} = 2$; Section 4.8 sweeps $w_{\text{pr}} \in [1.0, 2.4]$ to locate the precipitation/wind trade-off.

Monitoring for mode collapse. Even with this weighting, sparse heavy-tailed precipitation is an invitation to mode collapse: the model can minimise the weighted L2 by producing a slightly-wet constant field. During training we therefore log the wet-pixel fraction at thresholds $\{0.1, 1.0, 5.0\}$ mm h^{-1} and the 95th and 99th percentiles of the output distribution, and compare them to the same quantities on the training batch. A large divergence is a collapse warning.

doi:10.6342/NTU202601285

Chapter 4 Experiments and Results

This chapter presents the empirical evaluation of the MeanFlow residual of Chapter 3 against its two reference heads — the EDM diffusion baseline it sets out to accelerate and the FlowCast straight-line flow-matching baseline it generalises. Section 4.1 defines the experimental protocol and the matched-sample-budget convention. Section 4.2 defines the evaluation metrics. Section 4.3 reports the headline comparison on the 2022 validation year. Section 4.4 reports the inference-cost (wall-clock and NFE) ablation, where MeanFlow's one-to-two-evaluation operating point is the central result. Section 4.5 drills into precipitation skill by threshold and neighbourhood size; Section 4.6 reports autoregressive rollout error growth. Sections 4.7 and 4.8 report the `log1p`-encoding and precipitation-weight ablations. Section 4.9 places the heads against a legacy old-StormCast baseline. Section 4.10 presents qualitative case studies, and Section 4.11 concludes with a discussion; the failure modes observed in development are catalogued alongside the limitations in Section 5.2.

4.1 Experimental Protocol

Validation split. All reported metrics are computed on the cleaned 2022 full-year RWRF validation store (8,759 hourly samples; Section 3.2). This contrasts with the stale `valid_dates`

doi:10.6342/NTU202601285

shipped in `stormcast/config/inference/stormcast.yaml`, which we override explicitly in every run.

Matched training-sample budget. The fair-comparison unit is training samples seen, not optimiser steps, because the residual heads use different global batch sizes (EDM at batch 64, FlowCast at batch 96, MeanFlow at batch 112), which makes the step-to-sample conversion head-specific. We evaluate each head at the saved checkpoint nearest the $\approx 2\,$M-training-sample anchor; the MeanFlow checkpoint sits essentially on it. Holding training samples fixed means any quality difference reflects the generative trajectory and not a training-compute advantage.

Single-snapshot noise. Each validation point is a single snapshot. Adjacent validation logs drift by 10–$30\,\%$, so differences below $\approx 5\,\%$ on a single metric should not be over-interpreted; we call out only gaps that are large relative to this noise floor.

NFE budgets and samplers. The EDM baseline samples with its native 18-step Heun schedule (two model evaluations per step except the last, $2N - 1 = 35$ NFE per hourly step). FlowCast samples with K explicit Euler steps, K NFE; we use $K = 10$ in the headline scoreboard, the point at which the straight-line ODE is essentially converged (Section 4.4), and report the $K \in \{10, 15, 20\}$ cost ladder in the timing ablation. MeanFlow samples with K average-velocity segments (Eq. (3.14)), also exactly K NFE; its headline operating points are $K \in \{1, 2\}$. The scoreboard harness loads the two flow students from their online checkpoints in the same way, so the FlowCast-versus-MeanFlow comparison is symmetric end to end (the EMA shadow is used by the in-training validation sampler; Section 3.5).

doi:10.6342/NTU202601285

Rollouts and ensembling. The scoreboard metrics are computed from autoregressive rollouts driven by `compare_diffusion_vs_flowcast.py`, which denormalises every output (undoing standardisation and the `log1p` encoding) so that `qpepre` is scored in $\mathrm{mm\,h^{-1}}$ regardless of the on-disk encoding. Following the StormCast ensemble design [19], CRPS uses a **five-member** per-hour ensemble of independent noise seeds propagated autoregressively. Deterministic and categorical scores use the ensemble mean. Ten initial times are spaced evenly across the validation year, each rolled out autoregressively over a **six-hour** horizon ($+1\,\mathrm{h}\ \ldots\ +6\,\mathrm{h}$), matching StormCast's headline 1–6 h skill window.

Hardware. Training: multi-GPU single node. Evaluation: single GPU. The "Time/ Seq." column in the scoreboards is the wall-clock cost of generating one five-member six-hour rollout sequence on the evaluation GPU; the dedicated inference-timing ablation of Section 4.4 reports the operationally relevant cost at a 50-member, 24-hour configuration on a single NVIDIA RTX A6000.

4.2 Evaluation Metrics

All metrics are computed on denormalised fields with physical units (K, $\mathrm{m\,s^{-1}}$, $\mathrm{mm\,h^{-1}}$).

RMSE and MAE. For channel k,

$$\mathrm{RMSE}_k = \sqrt{\frac{1}{|\mathcal{T}|\,HW}\sum_{t\in\mathcal{T}}\sum_{i,j}\left(\hat{M}^{(k)}_{t,i,j} - M^{(k)}_{t,i,j}\right)^2}, \tag{4.1}$$

with MAE defined analogously.

doi:10.6342/NTU202601285

CRPS. The continuous ranked probability score evaluates the full predictive distribution of the small per-hour ensemble against the observation; lower is better. It rewards a forecast that is both sharp and calibrated, and is our primary probabilistic score.

Categorical precipitation scores. For threshold τ (mm h^{-1}), classify each pixel as wet ($\geq \tau$) or dry. With hits H, misses M, false alarms F:

$$\mathrm{CSI}(\tau) = \frac{H}{H+M+F}, \quad \mathrm{FAR}(\tau) = \frac{F}{H+F}, \quad \mathrm{HSS}(\tau) = \frac{2(H\,\mathrm{CN} - M\,F)}{(H+M)(M+\mathrm{CN}) + (H+F)(F+\mathrm{CN})}, \tag{4.2}$$

where CN is the correct-negative count. We report $\tau \in \{0.1, 1.0, 5.0, 10.0, 16.0\}$ mm h^{-1}; "-M" suffixes denote the mean over thresholds and "-P16" the $\tau = 16\,\mathrm{mm\,h^{-1}}$ heavy-rain point. CSI and HSS reward correct detection (higher is better); FAR penalises over-prediction (lower is better). To relax the single-pixel double-penalty, CSI/POD/FAR/HSS are scored with neighbourhood-max pooling: an event counts as forecast if any pixel within an $n \times n$ window exceeds τ. We evaluate at the four StormCast pooling windows [19] on the ≈ 2 km RWRF grid — 3 km (grid scale, 1×1 px), 15 km (7×7 px), 27 km (13×13 px) and 45 km (23×23 px) — and take the 15 km window as the headline.

Fractions Skill Score. With neighbourhood window n,

$$\mathrm{FSS}(\tau, n) = 1 - \frac{\sum_{i,j}(p^{\mathrm{fc}}_{i,j} - p^{\mathrm{ob}}_{i,j})^2}{\sum_{i,j}(p^{\mathrm{fc}}_{i,j})^2 + \sum_{i,j}(p^{\mathrm{ob}}_{i,j})^2}, \tag{4.3}$$

where $p_{i,j}$ is the fraction of τ-exceeding pixels in the $n \times n$ box centred at (i, j) [23]. FSS mitigates the double-penalty problem and is reported at the same four pooling windows; the heavy-rain point $\tau = 16\,\mathrm{mm\,h^{-1}}$ is denoted FSS-P16.

doi:10.6342/NTU202601285

Rollout metrics. Per-channel RMSE is additionally reported as a function of lead time $\ell \in \{1, \ldots, 6\}$ h to expose autoregressive error growth.

4.3 MeanFlow versus Diffusion and FlowCast: Headline Comparison

Table 4.1 is the central result of the thesis: MeanFlow (at one and two average-velocity segments) set against its two reference heads — the EDM diffusion baseline and the FlowCast straight-line baseline at $K = 10$ Euler steps — on the 2022 validation year, all at the matched ≈ 2 M-sample budget, sharing the same frozen regression mean and cleaned log1p dataset. Categorical scores are reported at the 15 km headline pooling window; Section 4.5 reports all four windows. Figure 4.1 visualises the same scoreboard.

Table 4.1: Headline comparison on the 2022 validation year (cleaned 192×96, log1p qpepre, ≈ 2 M training samples; five-member ensembles over ten initial times, $+6$ h rollouts). Categorical scores (15 km neighbourhood) and precipitation RMSE are in mm h^{-1} after denormalisation; `t2m` RMSE is in K and the winds in m s^{-1}. "-M" = mean over thresholds. ↑/↓ indicate the better direction; **bold** marks the best in each column. Time/Seq. is the wall-clock cost of one five-member six-hour rollout sequence on a single NVIDIA RTX A6000.

Method	Time/Seq. (s)	CRPS↓	CSI-M↑	HSS-M↑	FAR-M↓	RMSE t2m↓	RMSE u10↓	RMSE v10↓	RMSE qpepre↓
EDM diffusion (35 NFE)	11.23	0.5537	0.2622	0.2926	0.3417	0.8023	1.3513	1.6082	1.0292
FlowCast (K=10)	3.07	0.5518	0.2504	0.2889	0.3504	0.8498	1.2838	**1.4085**	**0.9590**
MeanFlow (K=1)	**0.91**	0.5501	**0.2689**	**0.3131**	0.2918	0.7498	1.3314	1.4844	0.9655
MeanFlow (K=2)	1.10	**0.5207**	0.2656	0.3128	**0.2563**	**0.7315**	**1.2651**	1.4286	0.9637

Read. At $K = 10$ FlowCast runs in 3.07 s per five-member sequence against 11.23 s for the 35-NFE EDM Heun sampler — a **3.7× speedup** — and at that budget matches or beats the diffusion baseline on most columns: CRPS is statistically indistinguishable (0.5518 vs 0.5537) and FlowCast leads on `u10` ($-5\,\%$), `v10` ($-12\,\%$), and `qpepre` ($-7\,\%$) RMSE,

doi:10.6342/NTU202601285

Figure 4.1: Per-metric comparison of the EDM diffusion baseline, FlowCast at $K = 10$, and MeanFlow at $K \in \{1, 2\}$ on the 2022 validation year (five-member ensembles, 15 km categorical window). FlowCast attains its skill at roughly a quarter of the diffusion wall-clock cost, MeanFlow at roughly a tenth.

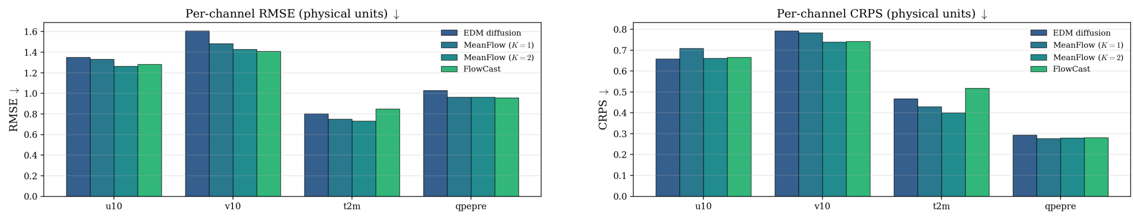
doi:10.6342/NTU202601285

conceding only `t2m` RMSE (0.8498 vs 0.8023) and the headline-window categorical hits (CSI-M 0.2622 vs 0.2504, HSS-M 0.2926 vs 0.2889). FAR is within noise at $15\,\text{km}$ but FlowCast is the lower-false-alarm head at the 3, 27, and $45\,\text{km}$ windows (Section 4.5); pushing it beyond $K = 10$ buys only wall-clock, the straight-line ODE being essentially converged by ≈ 10 steps (Section 4.4). MeanFlow compresses the cost axis by another large factor — $1.10\,\text{s}$ at $K = 2$ ($\approx 10.2\times$ cheaper than diffusion, $2.8\times$ cheaper than FlowCast) and $0.91\,\text{s}$ at $K = 1$ — and at $K = 2$ this comes with no quality penalty: it posts the best CRPS in the table (0.5207, a $\approx 6\,\%$ edge at the upper end of the noise floor), the best headline-window hit rates (CSI-M 0.2656, HSS-M 0.3128, both above the diffusion baseline), the lowest FAR-M (0.2563), and the best `u10` and `t2m` RMSE, conceding only a noise-floor margin on `v10`/`qpepre` RMSE to FlowCast. The $K = 1$ row trades a little of this for half the cost again: its `t2m` (0.7498) still leads the flow heads, but its CRPS (0.5501, $+5.6\,\%$ over $K = 2$) reflects the under-dispersion of Section 5.2 — a single average-velocity evaluation maps each noise draw almost onto the conditional mean, narrowing ensemble spread, a deficit that compounds autoregressively — which is why $K = 2$ is the recommended operating point. The picture is therefore no longer a simple speed-for-quality trade: at a tenth of the diffusion cost the average-velocity head is the strongest all-round candidate, with the more conservative FlowCast (lowest FAR across the wider neighbourhoods, column-leading `v10`/`qpepre` RMSE) a close alternative at $\sim 3\times$ higher cost.

Per-channel RMSE and CRPS. Figures 4.2 (a) and 4.2 (b) break the errors out by channel, and the two flow students split the wins: FlowCast keeps the lowest `v10` and `qpepre` RMSE (1.41, 0.96), while MeanFlow-$K2$ takes the lowest `u10` RMSE (1.27) and — decisively — the best `t2m` on both metrics (RMSE $0.7315\,\text{K}$, CRPS 0.4007), beating

doi:10.6342/NTU202601285

the diffusion baseline ($0.8023\,\mathrm{K}$ / 0.4681) by $\approx\ 9\,\%/14\,\%$. The `qpepre` and `v10` CRPS also tip to MeanFlow (best at $K1$ and $K2$ respectively, both inside the noise floor), while the diffusion baseline holds the lowest `u10` CRPS. The temperature result is the cleanest single-channel story: MeanFlow wins `t2m` outright at both K, evidence that FlowCast's `t2m` weakness belongs to its learned trajectory rather than the shared backbone, loss weights, or residual pipeline. The one place MeanFlow pays for a narrowed spread is the wind CRPS columns at $K = 1$, recovered at $K = 2$.



(a) Per-channel RMSE (physical units).

(b) Per-channel CRPS.

Figure 4.2: Per-channel deterministic (RMSE) and probabilistic (CRPS) errors for the diffusion baseline, FlowCast, and MeanFlow on the 2022 validation year. FlowCast holds the lowest `v10` and `qpepre` RMSE; MeanFlow-$K2$ wins temperature on both metrics and edges ahead on the precipitation and meridional-wind CRPS, with its narrowed spread showing up only in the K=1 wind CRPS.

4.4 Inference Cost: Wall-Clock and NFE Ablation

The defining property of a straight-line flow is that it integrates accurately in very few steps; the defining property of an average-velocity model is that it does not integrate at all. Tables 4.2 and 4.3 report the inference-timing ablation at an operationally realistic configuration — a 50-member, 24-hour ensemble forecast on a single NVIDIA RTX A6000 (the deployment GPU) and on a single NVIDIA H100 80GB — across the EDM Heun sampler, FlowCast at $K \in \{10, 15, 20\}$, and MeanFlow at $K \in \{1, 2\}$. This is the cost figure that matters for deployment: an ensemble convection-permitting cycle re-run every forecast hour. Figure 4.3 plots both devices' wall-clock on a log axis with each

doi:10.6342/NTU202601285

head's speedup over the EDM baseline annotated.

Table 4.2: Inference timing on a single NVIDIA RTX A6000 (50-member ensemble, 24 h autoregressive rollout, one CUDA warm-up forecast discarded). "s / forecast" is the wall-clock for one full 50-member 24 h ensemble forecast — the headline operational cost. "Speedup" is relative to the EDM diffusion sampler. "s / NFE" divides the per-forecast-hour cost by the per-step NFE; MeanFlow's higher per-NFE figure is fixed per-forecast overhead (I/O, denormalisation) amortised over only one–two evaluations, not a heavier network. **Bold** = best per column.

Method	NFE/step	s / forecast↓	Speedup↑	s / fcst-hr↓	s / member↓	s / NFE
EDM diffusion	35	1898.0	1.0×	1.582	37.96	0.044
FlowCast (K=10)	10	645.2	2.9×	0.538	12.90	0.054
FlowCast (K=15)	15	826.4	2.3×	0.689	16.53	0.046
FlowCast (K=20)	20	1086.3	1.7×	0.905	21.73	0.045
MeanFlow (K=1)	1	**191.9**	**9.9**×	**0.160**	**3.84**	0.160
MeanFlow (K=2)	2	239.1	7.9×	0.199	4.78	0.100

Table 4.3: Inference timing on a single NVIDIA H100 80GB HBM3 (same protocol as Table 4.2: 50-member ensemble, 24 h autoregressive rollout, one CUDA warm-up forecast discarded). Columns as in Table 4.2; "Speedup" is relative to the EDM diffusion sampler on the same device. **Bold** = best per column. The H100 roughly halves every wall-clock against the A6000 (A6000/H100 ≈ 1.8–$2.1\times$; see Figure 4.3), and the relative speedups over EDM grow slightly because the diffusion baseline's many evaluations dominate more on faster silicon.

Method	NFE/step	s / forecast↓	Speedup↑	s / fcst-hr↓	s / member↓	s / NFE
EDM diffusion	35	986.6	1.0×	0.822	19.73	0.023
FlowCast (K=10)	10	321.2	3.1×	0.268	6.42	0.027
FlowCast (K=15)	15	451.0	2.2×	0.376	9.02	0.025
FlowCast (K=20)	20	577.8	1.7×	0.482	11.56	0.024
MeanFlow (K=1)	1	**91.1**	**10.8**×	**0.076**	**1.82**	0.076
MeanFlow (K=2)	2	116.4	8.5×	0.097	2.33	0.048

Read. The ordering tracks the per-step NFE almost exactly. FlowCast at $K = 10$ generates a full 50-member 24-hour ensemble in 645 s ($\approx$ 10.8 min) against the diffusion sampler's 1898 s (≈ 31.6 min), a $2.9\times$ speedup; the $K = 15/20$ rungs scale linearly for no headline-skill gain, which is why $K = 10$ is the FlowCast operating point. MeanFlow cuts the same forecast to 239 s (≈ 4.0 min, $7.9\times$) at $K = 2$ and 192 s (≈ 3.2 min, $9.9\times$) at $K = 1$. Per evaluation the ODE heads cost ≈ 0.045 s–0.054 s, undercutting the Heun sampler's ≈ 0.044 s only marginally per evaluation but decisively in aggregate; Mean-

doi:10.6342/NTU202601285

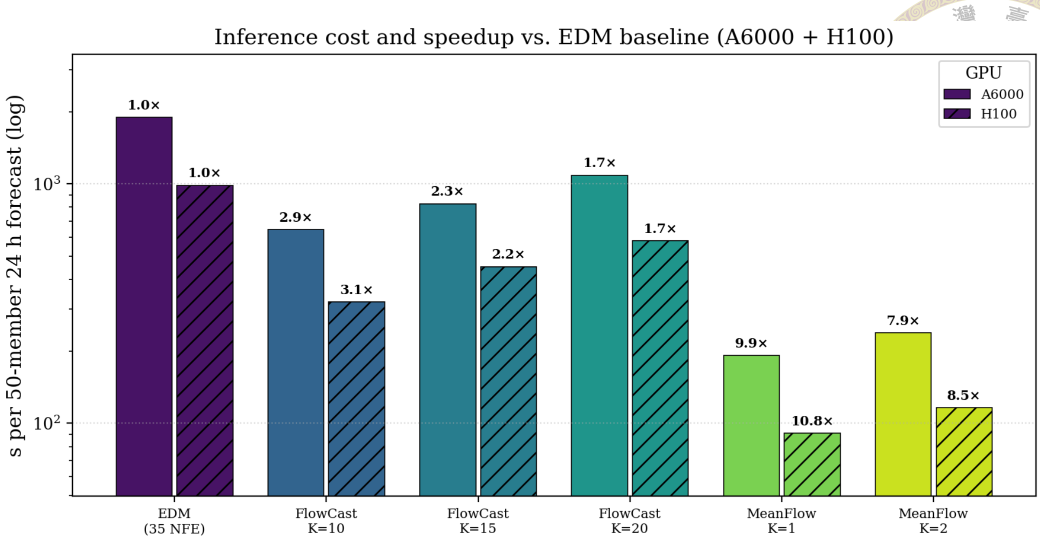


Figure 4.3: Wall-clock for one 50-member 24-hour ensemble forecast on a single NVIDIA RTX A6000 (solid) and a single NVIDIA H100 80GB (hatched), log axis, with each head's speedup over the 35-NFE EDM diffusion baseline (on the same device) annotated above its bar. On the A6000 FlowCast-$K10$ is $2.9\times$, MeanFlow-$K2$ $7.9\times$, and MeanFlow-$K1$ $9.9\times$ faster than EDM; on the H100 the corresponding factors are $3.1\times$, $8.5\times$, and $10.8\times$. The bars sample the viridis ramp from baseline (cool) to the average-velocity student (warm).

Flow's larger nominal s / NFE (0.10–0.16 s) is an amortisation artefact (fixed per-forecast overhead divided over one or two evaluations), not a heavier network — the backbone is the same SongUNet as FlowCast's.

Operational consequence. This is the cost that motivated the thesis (Section 1.4). A 50-member, 24-hour ensemble cycle issues 50×24 sequence-steps per forecast; at 35 NFE per step the diffusion sampler performs $50 \times 24 \times 35 = 42{,}000$ network evaluations per cycle, against $12{,}000$ for FlowCast-$K10$ and just $2{,}400$ for MeanFlow-$K2$ ($1{,}200$ at $K = 1$). Converting a configuration that the diffusion sampler places at half an hour of GPU time per cycle into one that completes in four minutes is what moves an ensemble convection-permitting forecast back inside an operational between-radar-volume budget. Crucially, these speedups are obtained without a second distillation training stage: they are intrinsic to the straight-line and average-velocity models.[1]

[1] The timing harness counts the EDM Heun sampler at 36 evaluations per step (two per Heun step, last corrector retained); the 35-NFE convention used elsewhere drops the final corrector. The one-evaluation

doi:10.6342/NTU202601285

Cross-hardware. The H100 leg (Table 4.3) confirms that the ranking is hardware-independent: the H100 roughly halves every wall-clock relative to the A6000 (A6000/H100 $= 1.8$–$2.1\times$ across all six configurations), but the relative speedups over the EDM baseline are preserved and even grow slightly — FlowCast-$K10$ goes from $2.9\times$ on the A6000 to $3.1\times$ on the H100, and MeanFlow-$K2$ from $7.9\times$ to $8.5\times$ ($K = 1$: $9.9\times \to 10.8\times$). The mechanism is that the diffusion baseline's 35 evaluations per step keep it compute-bound on faster silicon, while the one-to-two-evaluation MeanFlow head amortises a larger fixed-overhead fraction, so each hardware generation widens rather than narrows the gap. In absolute terms the H100 brings the MeanFlow-$K2$ cost to 116 s (≈ 1.9 min) and MeanFlow-$K1$ to 91 s (≈ 1.5 min) per 50-member 24-hour forecast, against the diffusion sampler's 987 s (≈ 16.4 min).

4.5 Precipitation Skill by Threshold and Neighbourhood

Mean categorical scores hide both the threshold dependence and the neighbourhood dependence that matter operationally. Figure 4.4 plots CSI, FAR, and HSS against the rainfall threshold at the headline window, and Figure 4.5 the heavy-rain FSS across the four pooling windows.

Neighbourhood dependence. Every categorical score improves monotonically as the pooling window widens from 3 km to 45 km, because the grid-scale double penalty relaxes. At $\tau = 0.1\,\mathrm{mm\,h^{-1}}$ the EDM baseline's CSI rises from 0.533 (3 km) to 0.720 (45 km) and FlowCast's from 0.546 to 0.738; the wet/dry discrimination the models actually have is a few-pixel-displaced one, which the grid-scale score under-credits. The 3 km (grid-scale)

difference does not affect any conclusion.

doi:10.6342/NTU202601285

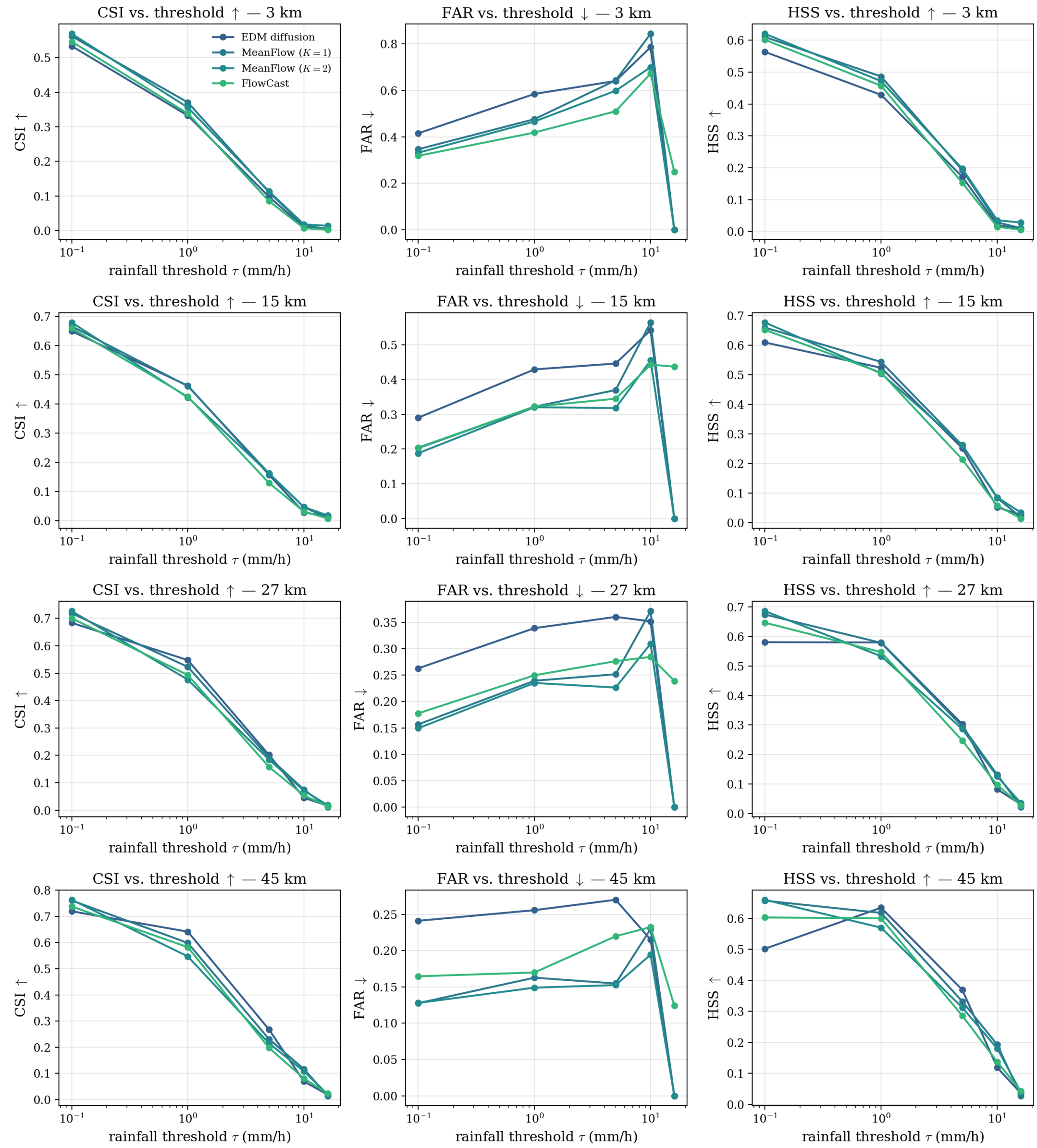


Figure 4.4: CSI, FAR, and HSS as a function of rainfall threshold for the diffusion baseline, FlowCast at $K = 10$, and MeanFlow at $K \in \{1, 2\}$ (five-member ensemble means, 15 km neighbourhood). All heads collapse toward zero CSI above $\approx 5\,\mathrm{mm\,h^{-1}}$ at this budget; the flow heads hold a false-alarm advantage at the light-to-moderate thresholds.

doi:10.6342/NTU202601285

categorical numbers collapse to near-zero at the heavy thresholds for all heads — this is the single-pixel artefact the StormCast pooling windows are designed to relax, not a model statement. Table 4.4 reports the threshold-averaged categorical scores across all four windows for every head.

Table 4.4: Threshold-averaged categorical precipitation skill (mean over $\tau \in \{0.1, 1, 5, 10, 16\}\,\mathrm{mm\,h^{-1}}$) at the four StormCast neighbourhood windows, with the heavy-rain FSS point (FSS-P16, $\tau = 16\,\mathrm{mm\,h^{-1}}$) alongside, for the legacy and cleaned heads (five-member ensemble means, 2022 validation year), grouped by neighbourhood window. **Bold** marks the best of the four cleaned heads for each metric within each window (↑ better for CSI-M/HSS-M/FSS-P16, ↓ better for FAR-M); the legacy EDM row is on a different 224×128 grid and is shown for context only, excluded from the bolding. Every score improves as the window widens; the $15\,\mathrm{km}$ block is the headline window of Table 4.1. FSS-P16 is near-zero for all heads (ensemble-mean smoothing of $\geq 16\,\mathrm{mm\,h^{-1}}$ extremes; Figure 4.5).

Window	Method	CSI-M↑	HSS-M↑	FAR-M↓	FSS-P16↑
3 km	Legacy EDM (224×128)	0.1008	0.1256	0.5768	0.0198
	EDM diffusion	0.1956	0.2384	0.4853	0.0088
	FlowCast (K=10)	0.1960	0.2462	0.4340	0.0053
	MeanFlow (K=1)	0.2127	0.2655	0.4620	0.0105
	MeanFlow (K=2)	**0.2141**	**0.2704**	**0.4192**	**0.0279**
15 km	Legacy EDM (224×128)	0.1448	0.1397	0.4110	0.0273
	EDM diffusion	0.2622	0.2926	0.3417	0.0076
	FlowCast (K=10)	0.2504	0.2889	0.3504	0.0038
	MeanFlow (K=1)	**0.2689**	**0.3131**	0.2918	0.0099
	MeanFlow (K=2)	0.2656	0.3128	**0.2563**	**0.0242**
27 km	Legacy EDM (224×128)	0.1791	0.1472	0.3349	0.0256
	EDM diffusion	0.2989	0.3151	0.2628	0.0065
	FlowCast (K=10)	0.2839	0.3129	0.2454	0.0040
	MeanFlow (K=1)	**0.3040**	**0.3403**	0.2038	0.0082
	MeanFlow (K=2)	0.2954	0.3332	**0.1841**	**0.0210**
45 km	Legacy EDM (224×128)	0.2312	0.1571	0.2570	0.0230
	EDM diffusion	0.3434	0.3320	0.1963	0.0053
	FlowCast (K=10)	0.3242	0.3337	0.1821	0.0043
	MeanFlow (K=1)	**0.3443**	**0.3654**	0.1349	0.0066
	MeanFlow (K=2)	0.3307	0.3522	**0.1247**	**0.0171**

Light to moderate rain. At $\tau = 0.1\,\mathrm{mm\,h^{-1}}$ ($15\,\mathrm{km}$) the heads are close on CSI (diffusion 0.650, FlowCast 0.660, MeanFlow-$K2$ 0.680), but the flow heads' false-alarm ratios are markedly lower (FlowCast 0.204, MeanFlow-$K2$ 0.188 vs diffusion 0.290), and the

doi:10.6342/NTU202601285

same ordering holds at $\tau = 1.0\,\mathrm{mm\,h^{-1}}$ (flow heads near FAR 0.32 vs diffusion 0.429). This is the quantitative form of the flow heads' "more conservative" behaviour — they do not paint speculative light rain across dry pixels — with MeanFlow-$K2$ the lowest-false-alarm head at the headline window.

Heavy rain. At $\tau = 5.0$ and $10.0\,\mathrm{mm\,h^{-1}}$ the heads converge to low CSI (≈ 0.13–0.17 at $5\,\mathrm{mm\,h^{-1}}$, MeanFlow-$K2$ marginally ahead), reflecting how hard intense convective cells are to place at the ≈ 2 M-sample budget — the clearest target for the longer-training and tail-loss experiments of Chapter 5. At $16\,\mathrm{mm\,h^{-1}}$ the comparison degenerates: with five-member ensemble means almost no pixel exceeds the threshold, so FSS-P16 ≤ 0.03 for all heads at every neighbourhood (Figure 4.5) — an ensemble-mean artefact (averaging suppresses extremes), which is why per-member heavy-rain scoring over larger ensembles is future work.

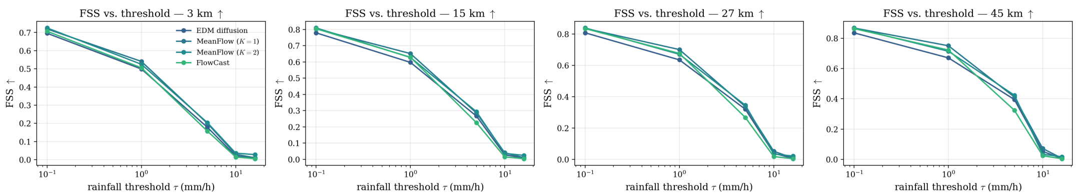


Figure 4.5: FSS at $\tau = 16\,\mathrm{mm\,h^{-1}}$ for the four pooling windows (3, 15, 27, 45 km), computed on five-member ensemble means, for all heads. The score is near-zero across the board: averaging members suppresses $\geq 16\,\mathrm{mm\,h^{-1}}$ extremes almost entirely, so this threshold mainly diagnoses ensemble-mean smoothing rather than model skill. Heavy-rain assessment at the member level requires probabilistic scoring (future work).

4.6 Autoregressive Rollout

Single-step skill understates operational risk because errors compound under autoregression. Figure 4.6 plots per-channel RMSE against lead time out to six hours, and Table 4.5 tabulates the same numbers at every lead.

doi:10.6342/NTU202601285

Table 4.5: Per-channel RMSE (physical units: $\mathrm{m\,s^{-1}}$ for winds, K for `t2m`, $\mathrm{mm\,h^{-1}}$ for `qpepre`) as a function of autoregressive lead time, $+1\,\mathrm{h} \ldots +6\,\mathrm{h}$, on the 2022 validation year (five-member ensemble means over ten initial times). **Bold** marks the best value per lead and channel. †Legacy `qpepre` is scored on the legacy 224×128 grid (raw $\mathrm{mm\,h^{-1}}$ encoding) and is not directly comparable to the cleaned 192×96 heads.

Lead (h)	Method	u10↓	v10↓	t2m↓	qpepre↓
1	Legacy EDM (224×128)	1.1364	1.1229	0.4899	**1.0441**†
	EDM diffusion	1.0035	1.0766	**0.4259**	1.1022
	FlowCast (K=10)	1.0094	1.0895	0.4708	1.0838
	MeanFlow (K=1)	1.0019	1.0849	0.4466	1.0725
	MeanFlow (K=2)	**1.0004**	**1.0568**	0.4299	1.0593
2	Legacy EDM (224×128)	1.5537	1.4332	0.6811	**1.1622**†
	EDM diffusion	1.2816	1.3563	0.5876	1.2361
	FlowCast (K=10)	1.2573	**1.2927**	0.6250	1.2254
	MeanFlow (K=1)	1.2777	1.3379	0.5881	1.2437
	MeanFlow (K=2)	**1.2235**	1.3088	**0.5521**	1.2489
3	Legacy EDM (224×128)	1.7302	1.6452	0.8474	**1.1179**†
	EDM diffusion	1.4005	1.6570	0.7477	1.2397
	FlowCast (K=10)	1.3141	**1.4876**	0.7839	1.2055
	MeanFlow (K=1)	1.3550	1.5578	0.7109	1.2164
	MeanFlow (K=2)	**1.2774**	1.5393	**0.6842**	1.2112
4	Legacy EDM (224×128)	1.8454	1.7200	0.9645	10.7571
	EDM diffusion	1.4003	1.7498	0.8679	1.3480
	FlowCast (K=10)	1.3180	**1.5556**	0.8998	**1.2718**
	MeanFlow (K=1)	1.3839	1.5981	0.7427	1.3221
	MeanFlow (K=2)	**1.2856**	1.5793	**0.7396**	1.3042
5	Legacy EDM (224×128)	2.0156	1.8685	1.0533	**1.1137**†
	EDM diffusion	1.5313	1.9133	0.9898	1.3238
	FlowCast (K=10)	**1.3689**	**1.5426**	1.0542	1.2434
	MeanFlow (K=1)	1.4866	1.6549	0.8764	1.2617
	MeanFlow (K=2)	1.3913	1.5668	**0.8464**	1.2515
6	Legacy EDM (224×128)	2.1399	2.1199	1.6099	**1.1005**†
	EDM diffusion	1.6711	2.2335	1.6008	1.2994
	FlowCast (K=10)	1.5574	**1.6805**	1.7272	1.2189
	MeanFlow (K=1)	1.6140	1.8869	**1.5690**	1.1979
	MeanFlow (K=2)	**1.5303**	1.7806	1.5737	1.2114

doi:10.6342/NTU202601285

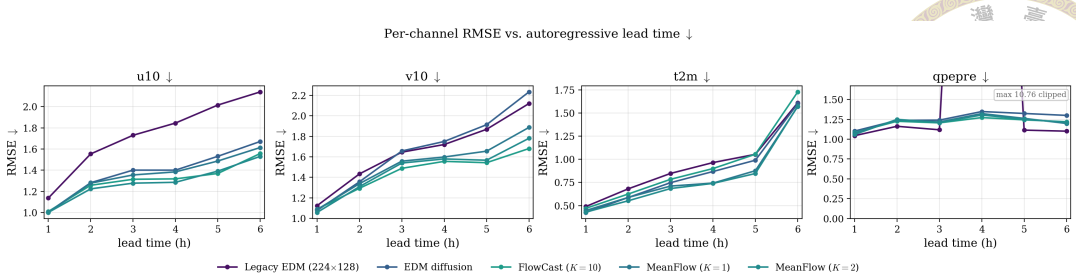


Figure 4.6: Per-channel RMSE as a function of autoregressive lead time (1–6 h) for the legacy baseline, the EDM diffusion baseline, FlowCast at $K = 10$, and MeanFlow at $K \in \{1, 2\}$. FlowCast tracks or improves on the diffusion baseline for the winds and precipitation across the horizon; MeanFlow holds the lowest `t2m` error of all heads across the horizon and the lowest `u10` at the longest lead, and neither flow head destabilises under rollout.

Read. All heads show the expected monotone error growth. For `v10` FlowCast stays well below the diffusion baseline at every lead ($1.68\,\mathrm{m\,s^{-1}}$ vs $2.23\,\mathrm{m\,s^{-1}}$ at $+6$ h), with MeanFlow-$K2$ just behind ($1.78\,\mathrm{m\,s^{-1}}$); for `u10` MeanFlow-$K2$ holds the lowest error at the longest lead ($+6$ h: $1.53\,\mathrm{m\,s^{-1}}$ against FlowCast's $1.56\,\mathrm{m\,s^{-1}}$ and the diffusion baseline's $1.67\,\mathrm{m\,s^{-1}}$) and leads most intermediate leads. The `qpepre` rollout RMSE does not diverge between the cleaned heads (≈ 1.2–$1.3\,\mathrm{mm\,h^{-1}}$ throughout) — the flow precipitation does not destabilise. Temperature, FlowCast's single-step weak spot, separates the heads most clearly: FlowCast's `t2m` error grows fastest (1.73 K at $+6$ h), whereas both MeanFlow settings hold `t2m` below the diffusion baseline across the whole horizon (1.57 K vs 1.60 K at $+6$ h), the rollout ordering following the single-step one of Section 4.3. The legacy baseline shows an isolated `qpepre` blow-up at $+4$ h ($\approx 10.8\,\mathrm{mm\,h^{-1}}$), an unphysical excursion none of the cleaned flow heads exhibits.

4.7 Ablation: `log1p` Precipitation Encoding

The single most consequential preprocessing decision is whether to store `qpepre` as raw $\mathrm{mm\,h^{-1}}$ or as $\log(1 + x)$ before standardisation. Table 4.6 reports the within-

doi:10.6342/NTU202601285

architecture A/B test (single-step standardised validation RMSE at $\approx 2\,\mathrm{M}$ samples) for the EDM and FlowCast heads. MeanFlow is absent from this table by necessity: only a log1p MeanFlow was trained, so its raw-mm/h cell does not exist. Because the mechanism identified below is head-agnostic — the badly-scaled raw channel contaminates the shared U-Net trunk — and MeanFlow shares that trunk, loss weighting, and the FlowCast objective on $75\,\%$ of its batch, we adopt log1p for MeanFlow without a separate ablation.

Table 4.6: Single-step standardised validation RMSE at $\approx 2\,\mathrm{M}$ samples, log1p versus raw-mm/h qpepre encoding. Values are dimensionless standard scores. **Bold** = better within each architecture pair on the non-qpepre channels. The qpepre column is not directly comparable across the encoding boundary (different units in standardised space); see the note.

Run	RMSE t2m	RMSE u10	RMSE v10	RMSE qpepre†
diffusion, log1p	**0.0994**	**0.3523**	**0.2762**	0.7865
diffusion, raw mm/h	0.1245	0.3955	0.3115	0.4156
FlowCast, log1p	**0.0835**	**0.2931**	0.2463	0.3851
FlowCast, raw mm/h	0.1118	0.3079	**0.2331**	0.3471

Read. The `log1p` encoding improves both heads on `t2m` and `u10` ($\sim$20–25 % and $\sim$5–11 %) and the diffusion baseline's `v10`; the lone exception is FlowCast's `v10`, tied within snapshot noise. The mechanism: the badly-scaled raw `qpepre` channel (std $9.12\,\mathrm{mm\,h^{-1}}$) propagates through the shared U-Net trunk and degrades the other channels, and compressing it to std ≈ 0.37 in log space removes that pressure. The benefit is head-independent, so we adopt `log1p` as canonical. The qpepre column (†) is not comparable across the encoding boundary (the two encodings carry different standardised units, and `expm1` does not commute with the root-mean-square); the $\mathrm{mm\,h^{-1}}$-correct comparison is the denormalised scoreboard of Section 4.3.

doi:10.6342/NTU202601285

4.8 Ablation: Precipitation Channel Weight

The precipitation weight w_{pr} (the last entry of `channel_weights`) trades precipitation skill against wind/temperature skill. Table 4.7 reports the full eight-point FlowCast sweep $w_{\text{pr}} \in [1.0, 2.4]$ (single-step standardised validation RMSE, log1p, $\approx 2\,\text{M}$ samples). Because `channel_weights` is applied only inside the training loss and not at evaluation, these RMSEs are directly comparable across rows without rescaling by w_{pr}. The sweep is FlowCast-only: w_{pr} is a training-time parameter, so a MeanFlow sweep would require seven additional training runs. MeanFlow instead adopts the sweep's winning all-round setting $w_{\text{pr}} = 2.0$ unchanged, which keeps the MeanFlow-versus-FlowCast headline comparison an A/B on the objective alone.

Table 4.7: FlowCast precipitation-weight sweep, single-step standardised validation RMSE (log1p, $\approx 2\,\text{M}$ samples). **Bold** = column minimum. $w_{\text{pr}} = 2.0$ is the headline run.

w_{pr}	RMSE t2m	RMSE u10	RMSE v10	RMSE qpepre
1.0	0.0994	0.3188	0.2326	0.5464
1.2	0.1032	0.2912	0.2110	**0.3771**
1.4	0.0890	**0.2733**	**0.2050**	0.3996
1.6	0.0930	0.3282	0.2391	0.6796
1.8	0.0983	0.2838	0.2075	0.4831
2.0 (headline)	**0.0835**	0.2931	0.2463	0.3851
2.2	0.0990	0.3178	0.2499	0.5178
2.4	0.0991	0.3075	0.2355	0.5515

Read. There is no clean monotone trend: both $w_{\text{pr}} = 1.4$ and $w_{\text{pr}} = 2.0$ beat their neighbours, while $w_{\text{pr}} = 1.6$ and $w_{\text{pr}} \geq 2.2$ are poor across the board. $w_{\text{pr}} = 1.4$ is the sweet spot for the wind fields, $w_{\text{pr}} = 2.0$ for `t2m`; on `qpepre` the minimum is a near-tie among $w_{\text{pr}} \in \{1.2, 1.4, 2.0\}$ inside the snapshot noise. The robust conclusions: some up-weighting helps but the spread is large and real (e.g. 0.3996 at $w_{\text{pr}} = 1.4$ vs 0.6796 at 1.6 on `qpepre`); $w_{\text{pr}} \approx 2.0$ is a reasonable all-round default (winning `t2m`, competitive

doi:10.6342/NTU202601285

on `qpepre`), which the canonical run adopts; and the wind/precipitation trade-off would justify $w_{\text{pr}} = 1.4$ if winds were the priority.

4.9 Comparison Against the Legacy Baseline

To place the cleaned-pipeline heads in context, Table 4.8 adds the legacy old-StormCast EDM baseline (the original 224×128 raw-qpepre model on the legacy dataset). Because the legacy grid and encoding differ, this is a pipeline-level rather than a controlled head-to-head comparison, and the absolute magnitudes are not directly comparable to the cleaned rows (the field of view differs by the centre crop).

Table 4.8: Legacy old-StormCast EDM baseline versus the cleaned-pipeline diffusion, FlowCast, and MeanFlow heads (categorical scores at the 15 km window, mm h^{-1}). The legacy leg uses a different grid/encoding, so absolute values are comparable only in magnitude (see Section 3.2).

Method	Time/Seq. (s)	CRPS↓	CSI-M↑	FAR-M↓	RMSE qpepre↓
Legacy EDM (raw, 224×128)	19.66	0.6348	0.1448	0.4110	1.4827
Cleaned EDM (35 NFE)	11.23	0.5537	0.2622	0.3417	1.0292
Cleaned FlowCast (K=10)	3.07	0.5518	0.2504	0.3504	**0.9590**
Cleaned MeanFlow (K=2)	**1.10**	**0.5207**	**0.2656**	**0.2563**	0.9637

Read. The cleaning + `log1p` + retraining pipeline is a large step up over the legacy baseline on every metric (CRPS $0.63 \rightarrow 0.55$, CSI-M $0.14 \rightarrow 0.26$ at the headline window, qpepre RMSE $1.48 \rightarrow 0.96$–1.03), independent of the generative head. The legacy-to-cleaned and cleaned-EDM-to-flow gaps are largely orthogonal: most of the headline improvement is the pipeline, and the flow heads' specific contribution is comparable-or-better skill at a fraction of the cost without a teacher — FlowCast at CRPS parity and the lowest precipitation RMSE, MeanFlow at the lowest cost ($\approx 18\times$ over legacy, $\approx 10\times$ over the cleaned diffusion baseline) and the best CRPS of any head (0.5207).

doi:10.6342/NTU202601285

4.10 Qualitative Case Studies

To complement the aggregate metrics we examine the single most precipitation-active initial time of the 2022 validation year — $t_0 = $ 2022-10-15 21:00 UTC, a heavy convective episode with a peak RWRF rain rate of $\approx 80\,\mathrm{mm\,h^{-1}}$ and $\approx 48\,\%$ of the domain wet at $\geq 1\,\mathrm{mm\,h^{-1}}$ — and roll every head forward autoregressively over the same $+1\,\mathrm{h} \ldots +6\,\mathrm{h}$ horizon used for the rollout metrics. Figure 4.7 stacks the precipitation fields (RWRF truth, legacy EDM, cleaned EDM diffusion, FlowCast, MeanFlow), and Figure 4.8 the zonal wind, all denormalised to physical units; the model rows load the matched ≈ 2 M-sample checkpoints used throughout the chapter.

Precipitation. The case renders the categorical trade-offs of Section 4.5 visible. The cleaned EDM diffusion head paints the broadest, most diffuse precipitation — moderate rain bleeding into dry pixels, the spatial signature of its higher false-alarm ratio. FlowCast is the most conservative: it keeps dry regions dry but visibly damps the intense cores, which wash out as the rollout advances — low FAR bought at the cost of heavy-rain hits. MeanFlow at two evaluations occupies the predicted middle ground: it retains markedly more high-intensity structure than FlowCast, re-forming bright $\geq 20\,\mathrm{mm\,h^{-1}}$ cells late in the rollout, while still suppressing the diffusion baseline's speculative light rain — and its two-evaluation fields are not visibly smoother than FlowCast's ten-step fields, so the average-velocity sampler pays no blurring penalty. None of the cleaned flow heads destabilises over the six hours, echoing the bounded rollout RMSE of Table 4.5.

doi:10.6342/NTU202601285

Wind. On `u10` (Figure 4.8) all heads track the broad-scale flow closely — the wind channels are far easier than precipitation — with the cleaned flow heads' fields lying closest to the RWRF target, consistent with their lower wind RMSE (Section 4.3) and the absence of wind-field drift under rollout.

4.11 Discussion

The headline question of the thesis is whether a single-run, teacher-free average-velocity residual can replace the EDM diffusion residual in a regional convection-permitting forecaster without losing the features that matter for Taiwan precipitation — and do so cheaply enough to deploy. The evidence at the matched $\approx 2\,$M-sample budget is yes: at $K \in \{1, 2\}$ network evaluations — roughly a tenth of the diffusion sampler's cost — MeanFlow reaches and, at $K = 2$, modestly exceeds the diffusion baseline's deterministic and probabilistic skill, posting the best CRPS (best in the study, though within the single-snapshot noise floor) and the best categorical hit rates of any head at the headline neighbourhood and the best temperature of any head, and runs a full 50-member 24-hour ensemble forecast in four minutes against the diffusion sampler's half hour. Its only genuine concession is the narrowed ensemble spread at $K = 1$, mostly closed by the second evaluation. The FlowCast straight-line baseline charts the intermediate point on the same trajectory axis: at $K = 10$ Euler steps it matches the diffusion baseline's CRPS, beats it on wind and precipitation RMSE, and runs $\approx 3.7\times$ faster — confirming that the straight-line path already recovers most of the diffusion skill before the average-velocity jump removes the last of the integration cost.

These conclusions are bounded by the single-snapshot budget, the single regression

doi:10.6342/NTU202601285

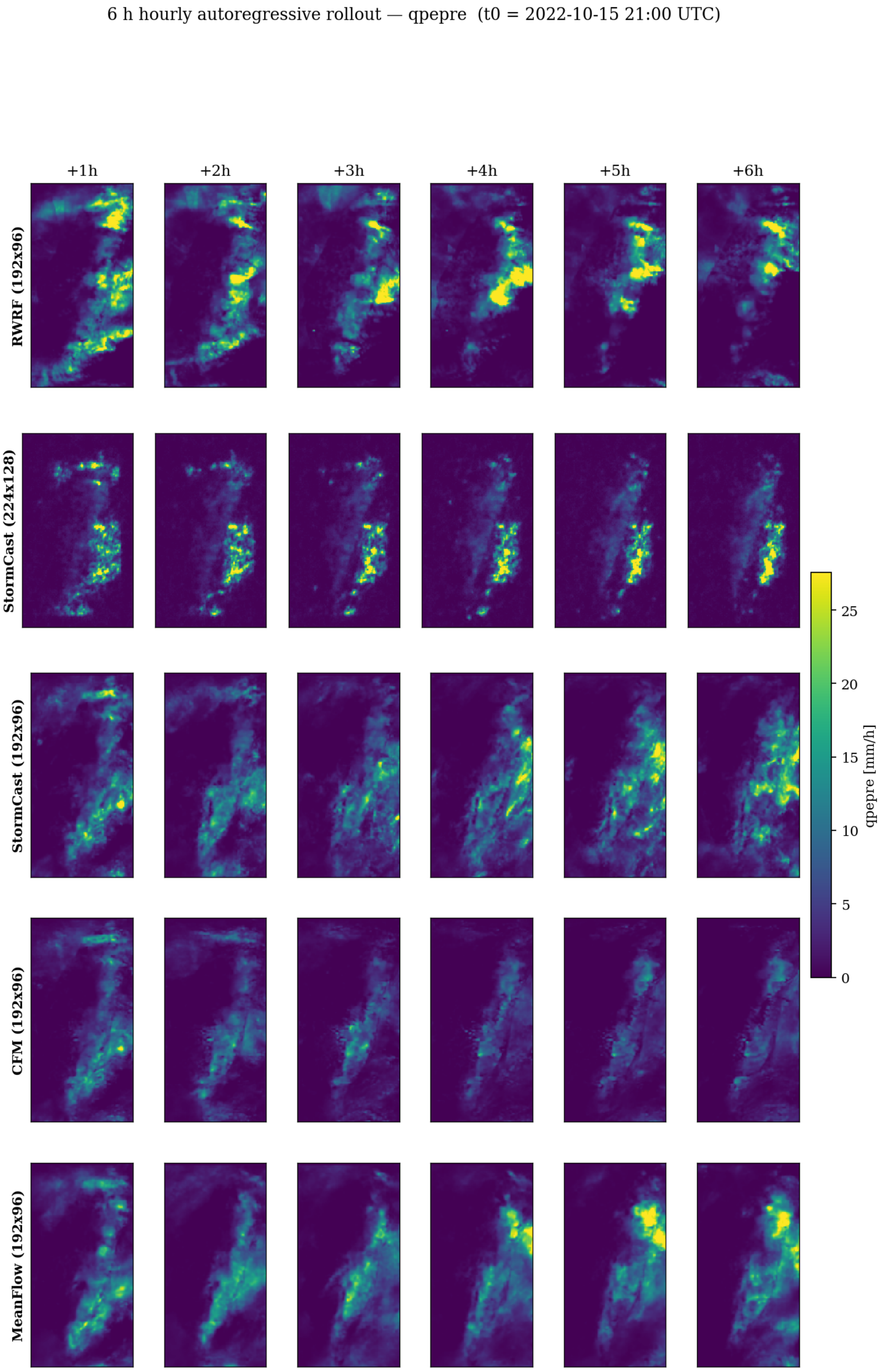


Figure 4.7: Six-hour autoregressive rollout of qpepre ($\mathrm{mm\,h^{-1}}$) for the heavy-rain initial time t_0 = 2022-10-15 21:00 UTC. Rows: RWRF/QPEPRE truth, legacy EDM (224×128), cleaned EDM diffusion (35 NFE), FlowCast (K=10), MeanFlow (K=2); columns: lead times $+1$ h … $+6$ h. The diffusion head spreads broad moderate rain, FlowCast damps the intense cores under rollout, and MeanFlow retains the most high-intensity structure of the flow heads at two network evaluations.

doi:10.6342/NTU202601285

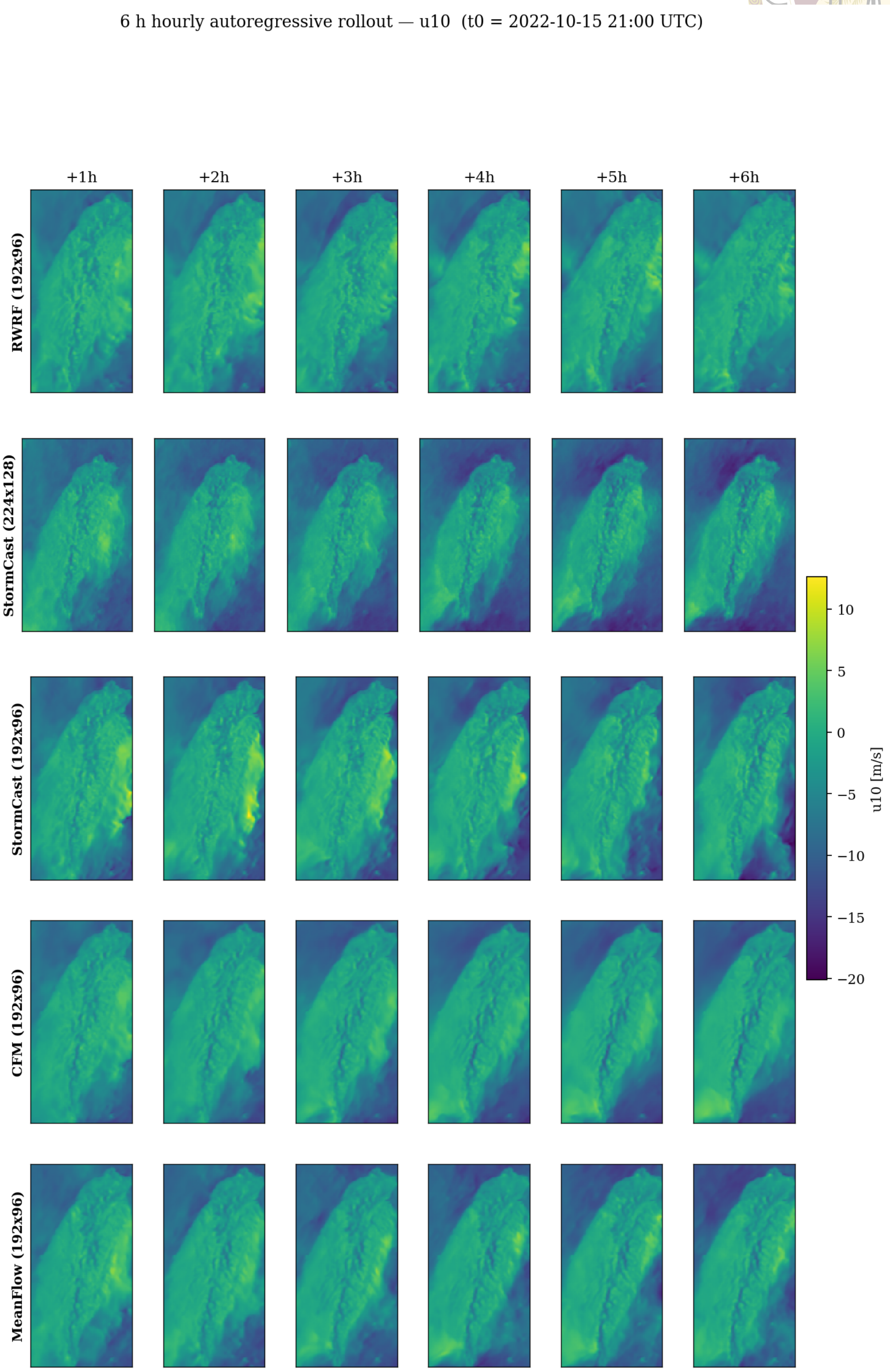


Figure 4.8: Six-hour autoregressive rollout of the 10 m zonal wind u10 ($m\,s^{-1}$) for the same initial time and row/column layout as Figure 4.7. All heads track the broad-scale wind field; the cleaned flow heads lie closest to the RWRF target with no rollout drift.

doi:10.6342/NTU202601285

mean and domain, and the precipitation-encoding gotcha of Section 4.7 (the regression mean and residual head must share the `log1p` encoding, or the residual distribution is silently corrupted); these are taken up in full, alongside the development-time failure modes, in Chapter 5, which lays out the concrete experiments still needed to turn this provisional result into a defensible claim.

doi:10.6342/NTU202601285

Chapter 5 Conclusion and Future Work

5.1 Summary of Contributions

This thesis asked whether a single-run, teacher-free average-velocity residual can replace the EDM diffusion residual in a regional, convection-permitting StormCast forecaster over the Taiwan domain — delivering the speed a curved diffusion ODE cannot, without sacrificing the sharpness and probabilistic calibration that motivated generative forecasting in the first place — and how few network evaluations such a residual ultimately needs. Starting from the StormCast two-stage decomposition (a frozen deterministic regression mean plus a generative residual head) trained on the Taiwan RWRF dataset, we adapted **MeanFlow** — which learns the average velocity over a whole time interval and transports the state across it in one or two network evaluations, with no integration at all — as the generative residual head. As baselines we retained the EDM diffusion residual it sets out to accelerate and **FlowCast**, a Conditional-Flow-Matching head that learns the straight-line instantaneous velocity field and samples it with a handful of explicit Euler steps and that MeanFlow generalises exactly. All three were compared head-to-head on the full 2022 hourly validation year.

doi:10.6342/NTU202601285

The concrete contributions were enumerated in Section 1.6; the headline finding is that, at a matched ≈ 2 M-sample budget, MeanFlow reaches the diffusion baseline's deterministic and categorical skill class at $K \in \{1, 2\}$ evaluations — roughly a tenth of the diffusion sampler's cost — posting the best CRPS (best in the study, though within the single-snapshot noise floor), the best categorical precipitation hit rates (CSI/HSS) at the headline neighbourhood, and the best temperature accuracy of any head, conceding only a spread-driven ensemble-calibration gap at $K = 1$ that the second evaluation closes. The FlowCast baseline meanwhile matches the diffusion CRPS, beats it on wind and precipitation RMSE and on false-alarm ratio at the wider neighbourhoods, and runs $\approx 3.7\times$ faster at 10 Euler steps — the intermediate point on the same trajectory axis.

5.2 Limitations

Several limitations — together with the failure modes observed during development — should be kept in mind when interpreting these results.

Single training snapshot. The headline comparison is taken at a single ≈ 2 M-sample checkpoint per head, where single-snapshot validation noise is 10–30 % (MeanFlow's checkpoint sits essentially on the anchor; Section 4.1). The canonical FlowCast run continues to ≈ 13.4 M samples, and the diffusion baseline likewise has more training available; a converged-budget comparison is not yet in hand.

Single regression mean and single domain. All three heads are anchored to one frozen regression mean trained on one regional domain (Taiwan RWRF). We cannot assess transfer to other convection-permitting domains (HRRR/CONUS StormCast) without retrain-

doi:10.6342/NTU202601285

ing, nor disentangle how much of the flow heads' behaviour depends on the specific quality of this regression mean.

Heavy-rain detection. The flow heads' conservative, low-false-alarm behaviour costs pixel-exact hits at the heavier rainfall thresholds. The neighbourhood FSS recovers part of this, suggesting a displacement rather than a pure miss, but the deterministic heavy-rain CSI is a genuine weakness at the current budget — and at the heaviest threshold, five-member ensemble-mean scoring degenerates for every head (averaging suppresses extremes), so the experiment cannot yet rank heads there at all.

Limited probabilistic evaluation. CRPS is computed from five-member per-hour ensembles — a small, operationally realistic ensemble in the StormCast tradition — but a large-ensemble calibration study (rank histograms, spread–skill at 50–100 members), which a cheap few-step sampler makes affordable, has not been run.

Under-dispersion at low NFE (MeanFlow). At $K = 1$ MeanFlow's deterministic skill is intact but its ensemble spread narrows — a single average-velocity jump lands each noise draw nearly on the conditional mean — so its aggregate CRPS lags its own $K = 2$ setting, most visibly on the wind channels, and the deficit compounds under autoregression: over the six-hour rollout the $K = 1$ CRPS (0.5501) trails $K = 2$ (0.5207) by $\approx 5.6\,\%$; at $K = 2$ MeanFlow in fact attains the best aggregate CRPS of all heads (0.5207 against the ODE heads' $0.5518/0.5537$), so no residual gap against them remains. The second evaluation restores most of the spread, which is why $K = 2$ is the recommended operating point; a per-member rank-histogram diagnosis has not yet been run.

doi:10.6342/NTU202601285

No distilled-diffusion comparator. We compare the flow heads against the full 35-NFE diffusion sampler, not against a distilled few-step diffusion student. MeanFlow demonstrates that the 1–2-NFE regime is reachable in a single run with no teacher; whether distillation could carry the diffusion baseline's heavy-rain CSI into that same regime is an open question this thesis does not answer.

FlowCast `t2m` rollout drift. FlowCast's temperature error grows fastest of the cleaned heads under rollout, reaching $1.73\,\mathrm{K}$ at $+6\,\mathrm{h}$ against the diffusion baseline's $1.60\,\mathrm{K}$ (Figure 4.6) — the autoregressive signature of its weaker single-step `t2m`. Notably MeanFlow does not inherit this drift despite sharing FlowCast's backbone and loss weights (its `t2m` stays below the diffusion baseline at $1.57\,\mathrm{K}$), so the drift is a property of the learned trajectory rather than the architecture; a per-channel weight that does not starve `t2m`, or a short teacher-forcing warmup, is the natural fix.

EMA evaluation pitfall. Loading the online `checkpoint_*.pt` training-state dump instead of the EMA shadow (`ema_state.pt`) produces visibly noisier flow-head samples. This is an evaluation pitfall rather than a model failure: the head-to-head harness loads the online `.mdlus` checkpoints for FlowCast and MeanFlow symmetrically, so the A/B comparison is unaffected, and the EMA-versus-online gap shrinks as training proceeds.

5.3 Future Work

The most promising directions, several of which are scoped concretely in the accompanying project to-do, are:

doi:10.6342/NTU202601285

1. **Train to convergence and re-score.** Run all three heads to a matched, converged sample budget and repeat the full metric suite, smoothing over several validation snapshots to beat down the $10–30\,\%$ noise. This is the single most important step to turn the provisional headline into a defensible claim.

2. **Isolate the loss recipe.** Re-train the EDM diffusion baseline with the same rain-aware channel weight w_{pr} the flow heads use, splitting the recipe's contribution from the generative objective's. The speed and NFE results are untouched by this experiment; the skill gaps are its target.

3. **Close the heavy-rain gap.** Test an explicit tail-quantile or focal loss on `qpepre`, a higher w_{pr}, and a stronger `asinh`/`log1p` dynamic-range transform, targeting the $5–10\,\mathrm{mm\,h^{-1}}$ CSI deficit without inflating false alarms.

4. **Restore low-NFE rollout spread.** MeanFlow under-disperses at $K = 1$ and the deficit compounds over autoregressive rollouts (a residual CRPS gap survives even at $K = 2$); candidate fixes — a stochastic single-segment sampler, temperature on the noise draw, per-step noise re-injection during rollout, or a small spread-aware loss term — would make the true 1–2-NFE regime probabilistically competitive end to end, diagnosed with per-member rank histograms.

5. **Large-ensemble calibration.** Exploit the flow heads' low per-sample cost to run 50–100-member ensembles and report CRPS, rank histograms, and spread–skill — the uncertainty-quantification payoff that cheap sampling unlocks. At MeanFlow's two evaluations, a 100-member 24-hour cycle costs $4{,}800$ network evaluations — about a ninth of what the 35-NFE diffusion sampler needs for a 50-member one — so ensemble size, not the sampler, becomes the budget knob.

doi:10.6342/NTU202601285

6. **Transfer to other domains.** Apply the same single-run flow recipes (FlowCast and MeanFlow) to another convection-permitting regression mean (e.g. an HRRR-trained StormCast) to measure how much of the pipeline is Taiwan-specific.

5.4 Closing Remarks

Generative modelling is almost certainly the right inductive bias for short-range convective-scale forecasting; the obstacle to deploying it has been the multi-step diffusion sampler. This thesis shows that, on a real regional dataset, the path a generative model takes from noise to weather is a dial worth turning twice — first straightened by a single-run straight-line flow residual (most of the diffusion skill at roughly a quarter of the cost, with no teacher), then jumped by an average-velocity residual that compresses the sampler to one or two evaluations, a tenth of the diffusion cost, at the study's best CRPS and categorical precipitation hit rates and trading only a modest one-evaluation spread gap. The remaining gaps — heavy-rain hits, FlowCast's `t2m` drift, one-evaluation spread, convergence at a larger budget — are concrete and addressable. Making sharp, probabilistic, regional convective forecasting fast enough to run between radar volumes is within reach, and single-run flow residuals — straight-line or average-velocity — are a credible path to it.

doi:10.6342/NTU202601285

References

[1] M. Andrychowicz, L. Espeholt, D. Li, S. Merchant, A. Merose, F. Zyda, S. Agrawal, and N. Kalchbrenner. Deep learning for day forecasts from sparse observations. arXiv preprint arXiv:2306.06079, 2023.

[2] K. Bi, L. Xie, H. Zhang, X. Chen, X. Gu, and Q. Tian. Accurate medium-range global weather forecasting with 3D neural networks. Nature, 619:533–538, 2023.

[3] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In Advances in Neural Information Processing Systems (NeurIPS), 2018.

[4] L. Espeholt, S. Agrawal, C. K. Sønderby, M. Kumar, J. Heek, C. Bromberg, C. Gazen, R. Carver, M. Andrychowicz, J. Hickey, A. Bell, and N. Kalchbrenner. Deep learning for twelve hour precipitation forecasts. Nature Communications, 13, 2022.

[5] Z. Gao, X. Shi, H. Wang, Y. Zhu, Y. Wang, M. Li, and D.-Y. Yeung. Earthformer: Exploring space-time transformers for earth system forecasting. In Advances in Neural Information Processing Systems (NeurIPS), volume 35, pages 25390–25403, 2022.

[6] Z. Gao, X. Shi, B. Han, H. Wang, X. Jin, D. Maddix, Y. Zhu, M. Li, and Y. Wang.

doi:10.6342/NTU202601285

PreDiff: Precipitation nowcasting with latent diffusion models. In Advances in Neural Information Processing Systems (NeurIPS), volume 36, pages 78621–78656, 2023.

[7] Z. Geng, M. Deng, X. Bai, J. Z. Kolter, and K. He. Mean flows for one-step generative modeling. arXiv preprint arXiv:2505.13447, 2025.

[8] J. Gong, L. Bai, P. Ye, W. Xu, N. Liu, J. Dai, X. Yang, and W. Ouyang. CasCast: Skilful high-resolution precipitation nowcasting via cascaded modelling. In International Conference on Machine Learning (ICML), 2024.

[9] H. Hersbach, B. Bell, P. Berrisford, S. Hirahara, A. Horányi, J. Muñoz-Sabater, J. Nicolas, C. Peubey, R. Radu, D. Schepers, A. Simmons, C. Soci, S. Abdalla, X. Abellan, G. Balsamo, P. Bechtold, G. Biavati, J. Bidlot, M. Bonavita, G. D. Chiara, P. Dahlgren, D. Dee, M. Diamantakis, R. Dragani, J. Flemming, R. Forbes, M. Fuentes, A. Geer, L. Haimberger, S. Healy, R. J. Hogan, E. Hólm, M. Janisková, S. Keeley, P. Laloyaux, P. Lopez, C. Lupu, G. Radnoti, P. de Rosnay, I. Rozum, F. Vamborg, S. Villaume, and J.-N. Thépaut. The ERA5 global reanalysis. Quarterly Journal of the Royal Meteorological Society, 146(730):1999–2049, 2020.

[10] J. Ho and T. Salimans. Classifier-free diffusion guidance. arXiv preprint arXiv:2207.12598, 2022.

[11] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In Advances in Neural Information Processing Systems (NeurIPS), 2020.

[12] T. Karras, M. Aittala, T. Aila, and S. Laine. Elucidating the design space of diffusion-based generative models. In Advances in Neural Information Processing Systems (NeurIPS), 2022.

doi:10.6342/NTU202601285

[13] R. Lam, A. Sanchez-Gonzalez, M. Willson, P. Wirnsberger, M. Fortunato, F. Alet, S. Ravuri, T. Ewalds, Z. Eaton-Rosen, W. Hu, A. Merose, S. Hoyer, G. Holland, O. Vinyals, J. Stott, A. Pritzel, S. Mohamed, and P. Battaglia. Learning skillful medium-range global weather forecasting. Science, 382(6677):1416–1421, 2023.

[14] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In International Conference on Learning Representations (ICLR), 2023.

[15] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In International Conference on Learning Representations (ICLR), 2023.

[16] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu. DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. In Advances in Neural Information Processing Systems (NeurIPS), 2022.

[17] M. Mardani, N. Brenowitz, Y. Cohen, J. Pathak, C.-Y. Chen, C.-C. Liu, A. Vahdat, K. Kashinath, J. Kautz, and M. Pritchard. Residual corrective diffusion modeling for km-scale atmospheric downscaling. arXiv preprint arXiv:2309.15214, 2024.

[18] J. Pathak, S. Subramanian, P. Harrington, S. Raja, A. Chattopadhyay, M. Mardani, T. Kurth, D. Hall, Z. Li, K. Azizzadenesheli, P. Hassanzadeh, K. Kashinath, and A. Anandkumar. FourCastNet: A global data-driven high-resolution weather model using adaptive Fourier neural operators. arXiv preprint arXiv:2202.11214, 2022.

[19] J. Pathak, Y. Cohen, P. Garg, P. Harrington, N. Brenowitz, D. Durran, M. Mardani, A. Vahdat, S. Xu, K. Kashinath, and M. Pritchard. Kilometer-scale convec-

doi:10.6342/NTU202601285

tion allowing model emulation using generative diffusion modeling. arXiv preprint arXiv:2408.10958, 2024.

[20] I. Price, A. Sanchez-Gonzalez, F. Alet, T. R. Andersson, A. El-Kadi, D. Masters, T. Ewalds, J. Stott, S. Mohamed, P. Battaglia, R. Lam, and M. Willson. Probabilistic weather forecasting with machine learning. Nature, 637(8044):84–90, 2025.

[21] S. Ravuri, K. Lenc, M. Willson, D. Kangin, R. Lam, P. Mirowski, M. Fitzsimons, M. Athanassiadou, S. Kashem, S. Madge, R. Prudden, A. Mandhane, A. Clark, A. Brock, K. Simonyan, R. Hadsell, N. Robinson, E. Clancy, A. Arribas, and S. Mohamed. Skilful precipitation nowcasting using deep generative models of radar. Nature, 597:672–677, 2021.

[22] B. P. Ribeiro and J. F. Pucer. FlowCast: Advancing precipitation nowcasting with conditional flow matching. arXiv preprint arXiv:2511.09731, 2025. To appear, International Conference on Learning Representations (ICLR) 2026.

[23] N. M. Roberts and H. W. Lean. Scale-selective verification of rainfall accumulations from high-resolution forecasts of convective events. Monthly Weather Review, 136 (1):78–97, 2008.

[24] T. Salimans and J. Ho. Progressive distillation for fast sampling of diffusion models. In International Conference on Learning Representations (ICLR), 2022.

[25] A. Sauer, D. Lorenz, A. Blattmann, and R. Rombach. Adversarial diffusion distillation. arXiv preprint arXiv:2311.17042, 2023.

[26] C. K. Sønderby, L. Espeholt, J. Heek, M. Dehghani, A. Oliver, T. Salimans, S. Agrawal, J. Hickey, and N. Kalchbrenner. MetNet: A neural weather model for precipitation forecasting. arXiv preprint arXiv:2003.12140, 2020.

doi:10.6342/NTU202601285

[27] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In International Conference on Learning Representations (ICLR), 2021.

[28] Y. Song and P. Dhariwal. Improved techniques for training consistency models. arXiv preprint arXiv:2310.14189, 2023.

[29] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In International Conference on Learning Representations (ICLR), 2021.

[30] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever. Consistency models. In International Conference on Machine Learning (ICML), 2023.

[31] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio. Improving and generalizing flow-based generative models with mini-batch optimal transport. Transactions on Machine Learning Research, 2024.

[32] M. Veillette, S. Samsi, and C. Mattioli. SEVIR: A storm event imagery dataset for deep learning applications in radar and satellite meteorology. In Advances in Neural Information Processing Systems (NeurIPS), volume 33, pages 22009–22019, 2020.

doi:10.6342/NTU202601285

doi:10.6342/NTU202601285

Appendix A — Hyperparameters and Training Configuration

A.1 EDM Diffusion Baseline Hyperparameters

The EDM diffusion baseline uses the following parameters. FlowCast and MeanFlow share the data scale σ_{data} verbatim so that the standardised-residual statistics and per-channel loss scaling are identical across all three heads (Section 3.4):

Parameter	Value
σ_{min}	0.002
σ_{max}	80
σ_{data}	0.5
ρ	7
Sampler	Heun
N (steps)	18 ($2N-1 = 35$ NFE)
Global batch size	64
Optimiser	AdamW

doi:10.6342/NTU202601285

A.2 FlowCast (Conditional Flow Matching)

Hyperparameters copied from `stormcast/config/model/flowcast.yaml` and `stormcast/config/training/flowcast.yaml`:

doi:10.6342/NTU202601285

Parameter	Value
σ_{data}	0.5
σ_{path} (I-CFM path std)	0.01
t_ε (time clamp)	10^{-5}
Time embedding scale τ	1000
`diffusion_conditions`	`[state, regression, invariant]`
`regression_conditions`	`[state, background, invariant]`
Backbone `channel_mult`	$[1, 2, 2, 2, 2]$
Optimiser	AdamW
Adam (β_1, β_2)	$(0.9, 0.999)$
Learning rate	5×10^{-4}
Weight decay	1×10^{-4}
Warmup steps (linear)	4,000
Min LR ratio (cosine floor)	10^{-2}
Total training steps	4×10^5
Global batch size	96
Gradient clip (global L2)	1.0
EMA decay (fixed)	0.999
Channel weights β	$(1, 1, 1, w_{\text{pr}})$, $w_{\text{pr}} = 2$ (headline)
Validation sampler	Euler, $K = 10$
Inference solvers	`euler` / `midpoint`

doi:10.6342/NTU202601285

A.3 MeanFlow (Average-Velocity Flow Matching)

Hyperparameters copied from `stormcast/config/model/meanflow.yaml` and `stormcast/config/training/meanflow.yaml`. Everything shared with FlowCast (optimiser, schedule, conditioning, backbone, channel weights, EMA) is kept identical so the FlowCast-versus-MeanFlow comparison isolates the objective; only the rows below the divider are MeanFlow-specific:

doi:10.6342/NTU202601285

Parameter	Value
σ_{data}	0.5
t_ε (time clamp)	10^{-5}
State-time embedding scale τ (on r)	1000
`diffusion_conditions`	`[state, regression, invariant]`
`regression_conditions`	`[state, background, invariant]`
Backbone `channel_mult`	$[1, 2, 2, 2, 2]$
Optimiser / Adam (β_1, β_2)	AdamW / $(0.9, 0.999)$
Learning rate / weight decay	5×10^{-4} / 1×10^{-4}
Warmup steps (linear) / min-LR ratio	4,000 / 10^{-2}
Gradient clip (global L2)	1.0
EMA decay (fixed)	0.999
Channel weights β	$(1, 1, 1, 2)$
(r, t) sampler	two i.i.d. $\mathcal{U}(t_\varepsilon, 1 - t_\varepsilon)$, sorted $r \leq t$
MeanFlow ratio ρ_{MF} ($r < t$ fraction)	0.25
Adaptive weight exponent p	1.0
Adaptive weight stabiliser ε_w	10^{-3}
Interval embedding (on $t - r$)	32-dim sin / cos Fourier, freqs $\in [1, 10^3]$
σ_{path}	0 (pure interpolant)
Global batch size	112 (7 GPUs)
Validation sampler	average-velocity, $K = 2$
Inference NFE (headline)	$K \in \{1, 2\}$

doi:10.6342/NTU202601285

A.4 Ablation Grids

The two ablations of Chapter 4 vary a single knob each, holding the rest of the FlowCast config fixed (the w_{pr} and encoding grids are FlowCast/EDM-only; MeanFlow adopts the winning settings, Sections 4.7–4.8):

Ablation	Values swept
Precipitation encoding (Section 4.7)	`log1p` / raw $\mathrm{mm\,h^{-1}}$
Precipitation weight w_{pr} (Section 4.8)	$\{1.0, 1.2, 1.4, 1.6, 1.8, 2.0, 2.2, 2.4\}$
Inference steps K, both flow heads (Section 4.4)	$\{1, 2, 3, 4, 5, 6, 8, 10, 12, 16, 20, 25, 32, 40, 50\}$; scoreb

A.5 Per-Channel Dataset Statistics

High-resolution target statistics computed on the 2019-08 – 2021-12 training split (original store; the cleaned store re-derives statistics after the `log1p` encoding of `qpepre`):

Channel	Mean	Std
`t2m` (K)	295.98	5.61
`u10` ($\mathrm{m\,s^{-1}}$)	-1.58	3.84
`v10` ($\mathrm{m\,s^{-1}}$)	-2.65	5.66
`qpepre` ($\mathrm{mm\,h^{-1}}$, raw)	0.182	9.12
`qpepre` ($\log(1+x)$ space)	—	≈ 0.37

doi:10.6342/NTU202601285

Appendix B — Reproducibility

B.1 Code Availability

The full training and evaluation code for this thesis lives in the `Stormcast-Distilation` repository. The load-bearing entry points are:

- `stormcast/train.py` — regression and EDM diffusion baseline training (Hydra config selects the mode); `train_flowcast.py` and `train_meanflow.py` — the two flow students.

- `stormcast/utils/flowcast_loss.py` / `meanflow_loss.py` — the I-CFM and MeanFlow (JVP-bootstrapped, stop-gradient, adaptively weighted) losses, both sharing the per-channel weights.

- `stormcast/utils/flowcast_precond.py` / `meanflow_precond.py` — the velocity and two-time average-velocity wrappers.

- `stormcast/utils/nn.py` — `get_preconditioned_architecture`, `build_network_conditio`
  and the samplers `diffusion_model_forward` / `flowcast_model_forward` / `meanflow_model_for`

- `experiment_scripts/compare_diffusion_vs_flowcast.py` — the head-to-head rollout scoreboard harness of Chapter 4 (the MeanFlow leg is enabled with

doi:10.6342/NTU202601285

`--meanflow-checkpoint`); `flowcast_nfe_sweep.py` — the NFE-versus-quality sweep of Section 4.4.

- `experiment_scripts/make_thesis_figures.py` / `make_thesis_qualitative.sh` — regenerate every result figure of Chapter 4; all plotting reads its colours from `thesis_style.py` so the figure set stays on one palette.

B.2 Dataset and Checkpoints

Zarr store. The cleaned Taiwan RWRF dataset used for the reported experiments is the local Zarr store

```
exp_3_…_full_cleaned_4_27_2026/
```

with sub-groups `LowRes/`, `HighRes/`, and `invariants/`, a 192×96 grid, and `log1p`-encoded `qpepre` (Section 3.2).

Checkpoints (cleaned, log1p, ≈ 2 M-sample anchor).

- Regression mean: `StormCastUNet.0.8000.mdlus` (shared by all three heads).
- EDM diffusion baseline: the ≈ 2 M-training-sample `EDMPrecond` checkpoint.
- FlowCast: the ≈ 2 M-training-sample `FlowCastPrecond` checkpoint; the sibling `ema_state.pt` holds the EMA shadow used by the in-training validation sampler. The canonical FlowCast run continues to the ≈ 13.4 M-sample checkpoint.
- MeanFlow: the ≈ 2 M-training-sample `MeanFlowPrecond` checkpoint (batch 112) with its sibling `ema_state.pt`. The Chapter 4 harness loads the FlowCast and

doi:10.6342/NTU202601285

MeanFlow online checkpoints symmetrically.

B.3 Preprocessing Pipeline

The pipeline that assembles the Zarr store is described in Section 3.3.2; the two staged scripts live under `data_preprocessing/`: `nc_month_to_day/` splits monthly ERA5 NetCDF files into single-hour files, and `nc_to_zarr/` (driven by `config.yaml`) builds the file indices, precomputes the joint-validity mask, extracts and re-grids each source onto the shared Taiwan grid, accumulates single-pass (Welford) per-channel statistics, and writes a consolidated StormCast-style Zarr store whose on-disk layout is recorded in `zarr_metadata.json` and consumed by `data_loader_rwrf_era5_stable.Dataset`.

B.4 Commands

All commands launch from the `stormcast/` directory. The shipped configs point at stale cluster paths and outdated `valid_dates`; override `dataset.location`, the regression weights, and `dataset.valid_dates=['2022/01/01','2022/12/31']` on the local workstation. Each head is trained by its entry point (`train.py` for the regression mean and, with `--config-name=diffusion`, the EDM baseline; `train_flowcast.py`; `train_meanflow.py`), passing `model.regression_weights` and `dataset.location`; re-running the same command auto-resumes from the latest checkpoint plus `ema_state.pt`.

The head-to-head scoreboard, per-threshold, FSS, CRPS, rollout, and qualitative-panel outputs of Chapter 4 are produced by

```
bash experiment_scripts/run_main_experiment.sh
```


doi:10.6342/NTU202601285

which runs `compare_diffusion_vs_flowcast.py` (MeanFlow leg at `MEANFLOW_NFES="1 2"`) and writes `scoreboard.{md,csv}`, `per_threshold.csv`, `fss_p16.csv`, `rmse_per_step.csv`, and `crps_per_channel.csv` in physical units (qpepre in $\mathrm{mm\,h^{-1}}$ after inverse-`log1p`). The NFE-versus-quality sweeps of Section 4.4 are wrapped by `run_flowcast_nfe_sweep.sh` (`METHODS="flowcast meanflow"`), and every result figure is regenerated by

```
bash experiment_scripts/make_thesis_figures.sh      # CPU only
bash experiment_scripts/make_thesis_qualitative.sh  # one GPU, ~2 min
```

The matched $\approx 2\,$M-sample checkpoints must be passed explicitly to the sweep, as its FlowCast default points at the longer-trained $\approx 13.4\,$M-sample checkpoint, which is not comparable to the Chapter 4 scoreboard.

doi:10.6342/NTU202601285

Appendix C — Detailed Derivations

This appendix collects line-by-line derivations of the identities the method of this thesis rests on, without passing through to the literature. The goal is that any reader with a working knowledge of multivariate Gaussians and basic Itô calculus can reconstruct, from scratch, the score–denoiser (Tweedie) identity and the probability-flow ODE that define the EDM diffusion baseline, the Conditional Flow Matching theorem and its straight-line Independent-CFM instantiation (the FlowCast baseline), and finally the MeanFlow average-velocity identity at the centre of the thesis. The EDM preconditioning scalars and ρ-spaced noise schedule (baseline-only machinery, stated in full in Section 2.6 with the original reference) and the discrete-time DDPM/DDIM construction are omitted, as the method does not rely on them; the condensed treatment in Chapter 2 suffices for context. Notation in each section is local; all symbols have the same meaning as in Chapters 2–3 unless otherwise specified.

doi:10.6342/NTU202601285

C.1 Tweedie's Formula: Score and Denoiser

Let $x_0 \sim p_{\text{data}}$ and $x = x_0 + \sigma\epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. Write $p_\sigma(x) = \int p_{\text{data}}(x_0)\mathcal{N}(x; x_0, \sigma^2 I)\, dx_0$. Differentiating under the integral,

$$\nabla_x p_\sigma(x) = \int p_{\text{data}}(x_0)\, \nabla_x \mathcal{N}(x; x_0, \sigma^2 I)\, dx_0 = \int p_{\text{data}}(x_0)\, \frac{x_0 - x}{\sigma^2}\, \mathcal{N}(x; x_0, \sigma^2 I)\, dx_0.$$

Dividing by $p_\sigma(x)$,

$$\nabla_x \log p_\sigma(x) = \frac{1}{\sigma^2}\, \mathbb{E}[x_0 - x \mid x] = \frac{\mathbb{E}[x_0 \mid x] - x}{\sigma^2}.$$

Identifying the MMSE denoiser $D^\star(x; \sigma) = \mathbb{E}[x_0 \mid x]$ gives Tweedie's formula

$$\boxed{\nabla_x \log p_\sigma(x) = \frac{D^\star(x; \sigma) - x}{\sigma^2}.} \tag{C.1}$$

Because the Bayes-optimal denoiser is the conditional expectation, score matching and denoising regression are equivalent as learning objectives: fitting D_θ to minimise $\mathbb{E}\|D_\theta(x; \sigma) - x_0\|^2$ simultaneously learns an estimate of the score via (C.1).

C.2 Probability-Flow ODE from the VE-SDE

The variance-exploding forward SDE $dx = g(\sigma)\, dW$ with $g(\sigma) = \sqrt{d\sigma^2/dt}$ admits a deterministic probability-flow ODE that shares its time marginals p_σ: matching the Fokker–Planck equation of the SDE to the continuity equation of the candidate ODE $dx/dt = f(x, \sigma)$ [29] forces the drift $f = -\frac{1}{2} g^2\, \nabla_x \log p_\sigma$, which in σ-time ($g^2 =$

doi:10.6342/NTU202601285

$2\sigma\, d\sigma/dt$) collapses to the clean form

$$\boxed{\frac{dx}{d\sigma} = -\sigma\, \nabla_x \log p_\sigma(x).} \tag{C.2}$$

Combining (C.2) with Tweedie (C.1) gives the denoiser-parameterised PF-ODE

$$\frac{dx}{d\sigma} = \frac{x - D^\star(x;\sigma)}{\sigma}, \tag{C.3}$$

the form that first-order (Euler / DDIM) and second-order (Heun) integrators evaluate at inference.

C.3 CFM Theorem: Conditional = Marginal Gradient

Fix a Gaussian conditional probability path $p_t(x \mid z)$ indexed by a latent $z \sim q(z)$, with corresponding conditional vector field $u_t(x \mid z)$. The marginal flow-matching loss is

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t,p_t(x)}\big[\|v_\theta(x,t) - u_t(x)\|^2\big],$$

where $p_t(x) = \mathbb{E}_{q(z)}[p_t(x \mid z)]$ and $u_t(x) = \mathbb{E}[u_t(x \mid z) \mid x, t]$ is the marginal vector field. This loss is intractable because $u_t(x)$ requires marginalising over z. The conditional flow-matching loss replaces the marginal target with the conditional one:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,z,p_t(x|z)}\big[\|v_\theta(x,t) - u_t(x \mid z)\|^2\big].$$

Theorem. $\nabla_\theta \mathcal{L}_{\text{FM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{CFM}}(\theta)$.

doi:10.6342/NTU202601285

Proof. Expanding the squared norms and dropping θ-independent terms,

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t,p_t(x)}\big[\|v_\theta(x,t)\|^2 - 2\,v_\theta(x,t)^\top u_t(x)\big] + \text{const},$$

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,z,p_t(x|z)}\big[\|v_\theta(x,t)\|^2 - 2\,v_\theta(x,t)^\top u_t(x \mid z)\big] + \text{const}.$$

The first term is identical after applying the law of total expectation, $\mathbb{E}_{z,p_t(x|z)} = \mathbb{E}_{p_t(x)}$. For the second term,

$$\begin{aligned}\mathbb{E}_{t,z,p_t(x|z)}\big[v_\theta(x,t)^\top u_t(x \mid z)\big] &= \mathbb{E}_t\,\mathbb{E}_{p_t(x)}\big[v_\theta(x,t)^\top\,\mathbb{E}[u_t(x \mid z) \mid x,t]\big] \\ &= \mathbb{E}_{t,p_t(x)}\big[v_\theta(x,t)^\top u_t(x)\big],\end{aligned}$$

using $u_t(x) = \mathbb{E}[u_t(x \mid z) \mid x,t]$. Hence $\mathcal{L}_{\mathrm{FM}} = \mathcal{L}_{\mathrm{CFM}} + \text{const}$, and their gradients coincide. □

The upshot is that training against the conditional target is an unbiased (and computable) estimator of gradients of the marginal objective, and the latter is exactly the flow-matching objective whose ODE transports $p_0 \to p_1$.

C.4 Independent CFM: Straight-Line Path and Target

Take $z = (x_0, x_1)$ with $x_0 \sim \mathcal{N}(0, I)$ and $x_1 \sim p_{\text{data}}$, independent. Define the Gaussian probability path

$$p_t(x \mid x_0, x_1) = \mathcal{N}\big((1-t)x_0 + t x_1,\ \sigma^2_{\text{path}} I\big), \qquad t \in [0,1].$$

doi:10.6342/NTU202601285

Target vector field. Under the flow $\phi_t(x \mid x_0, x_1) = (1-t)x_0 + tx_1 + \sigma_{\text{path}}\,\eta$ with η fixed, the associated vector field is

$$u_t(x \mid x_0, x_1) = \frac{d}{dt}\phi_t = x_1 - x_0,$$

which is independent of x (and of t) — a straight-line constant-velocity interpolation from noise to data. Plugging into $\mathcal{L}_{\text{CFM}}$,

$$\mathcal{L}_{\text{I-CFM}}(\theta) = \mathbb{E}_{t,x_0,x_1,\eta}\big[\|v_\theta((1-t)x_0 + tx_1 + \sigma_{\text{path}}\eta,\ t) - (x_1 - x_0)\|^2\big]. \tag{C.4}$$

Why $\sigma_{\text{path}} > 0$. At $\sigma_{\text{path}} = 0$ the conditional path is a Dirac at each time, supported on the line segment between x_0 and x_1. In the marginalised view this is a measure-zero set, so the score of the marginal $p_t(x)$ is undefined on that set and the empirical loss gradient is extremely high-variance in high dimensions (Rectified-Flow pathology; see 15). Taking $\sigma_{\text{path}} > 0$ convolves the path with a spherical Gaussian of width σ_{path}, which regularises the target field onto a thin tube around the interpolant and smooths the loss landscape. Tong et al. [31] report $\sigma_{\text{path}} \in [10^{-3}, 10^{-1}]$ all work in practice; FlowCast and our adaptation use $\sigma_{\text{path}} = 0.01$.

Compared to diffusion. The straight-line ODE of I-CFM and the PF-ODE of a score-based diffusion both transport $\mathcal{N}(0, I) \to p_{\text{data}}$, but their trajectories differ. The diffusion PF-ODE is curved: its Jacobian sees the local score, which in turn depends on the local data density, and its integration error grows quickly as the step count shrinks. The I-CFM ODE has constant velocity $x_1 - x_0$ along each conditional trajectory, so the marginal field is the closest approximation (in L^2) to a linear transport available — which is exactly why a single Euler step is already competitive.

doi:10.6342/NTU202601285

C.5 FlowCast's Standardised-Residual Adaptation

The adaptation used in Chapter 3 applies I-CFM to the StormCast residual pipeline with the following changes to the textbook I-CFM of (C.4):

1. **Residual target.** x_1 is the standardised residual R_t/σ_{data} rather than the raw field. Standardisation absorbs the per-channel scale so the noise $x_0 \sim \mathcal{N}(0, I)$ is comparable in magnitude to the data target.

2. **Condition bundle.** v_θ is conditioned on $c_t = \text{concat}(M_{t-1}, \mu_t, I)$ (the low-resolution background S_t is omitted from the direct path and reaches the velocity field only through the regression mean μ_t; see Section 3.5). This keeps the FlowCast residual anchored to exactly the same μ_t as the EDM diffusion baseline. No extra VAE encoder is inserted, unlike the original FlowCast which works in latent space.

3. **Time rescaling.** The flow time $t \in [0, 1]$ is multiplied by $\tau = 1000$ before entering the SongUNet time embedding, because the positional-embedding frequencies of the pretrained architecture were tuned for the EDM $\log \sigma$ range.

4. **Channel-weighted loss.** The pointwise MSE is the channel-weighted L2 norm $\|\cdot\|_{2,\beta}$ of Section 3.7, which up-weights the sparse, heavy-tailed `qpepre` channel relative to the three near-Gaussian fields.

5. **Endpoint clamp.** Training samples $t \sim \mathcal{U}(t_\varepsilon, 1 - t_\varepsilon)$ with $t_\varepsilon = 10^{-5}$. At $t = 0$ or $t = 1$ the conditional path collapses to a point mass and the Monte-Carlo loss gradient is unbounded; the clamp prevents numerical blow-ups at negligible bias cost.

doi:10.6342/NTU202601285

The resulting training loop differs from I-CFM only in these five bookkeeping details; the underlying theorem (§C.3) and path derivation (§C.4) carry over unchanged, and the sampler (3.7) is the explicit Euler integrator of the straight-line ODE.

C.6 The MeanFlow Average-Velocity Identity

This is the derivation behind the method of this thesis: the MeanFlow identity [7], transcribed to the noise-at-0, data-at-1 convention used throughout (Section 3.6). It turns an intractable path integral into a pointwise relation the network can be regressed against.

Setup. Work in the standardised residual space of §C.5: $x_1 = R_t/\sigma_{\text{data}}$ is the data endpoint, $x_0 \sim \mathcal{N}(0, I)$ the prior, and the linear interpolant $z_s = (1-s)\,x_0 + s\,x_1$ runs from noise at $s = 0$ to data at $s = 1$ along the trajectory $\dot{z}_s = v(z_s, s)$ with v the (marginal) velocity field of §C.4. For two times $r \leq t$ define the average velocity

$$u(z_r, r, t) \;\triangleq\; \frac{1}{t-r}\int_r^t v(z_\tau, \tau)\, d\tau, \tag{C.5}$$

the displacement of the flow over $[r, t]$ per unit time. Two immediate properties follow. First, the limit as $t \to r$ is the instantaneous velocity, $\lim_{t\to r} u(z_r, r, t) = v(z_r, r)$, by the fundamental theorem of calculus. Second, multiplying by $(t - r)$ makes transport across the interval a single algebraic step,

$$z_t = z_r + (t-r)\, u(z_r, r, t), \tag{C.6}$$

exactly and with no solver — one evaluation at $(r, t) = (0, 1)$ maps noise to data. The catch is that (C.5) is an intractable integral, so u cannot be regressed directly.

doi:10.6342/NTU202601285

Differentiating away the integral. Clear the denominator in (C.5),

$$(t-r)\,u(z_r,r,t) \;=\; \int_r^t v(z_\tau,\tau)\,d\tau,$$

and differentiate both sides with respect to the <u>state</u> time r, holding the <u>horizon</u> time t fixed. Along the trajectory the state moves with $dz_r/dr = v(z_r,r)$ and $dt/dr = 0$. The left-hand side, by the product rule, is

$$\frac{d}{dr}\Big[(t-r)\,u\Big] = -\,u(z_r,r,t) + (t-r)\,\frac{d}{dr}u(z_r,r,t).$$

The right-hand side, by the fundamental theorem of calculus applied to the <u>lower</u> limit (hence the leading minus sign), is

$$\frac{d}{dr}\int_r^t v(z_\tau,\tau)\,d\tau = -\,v(z_r,r).$$

Equating the two and cancelling the $-u$ against $-v$ rearrangement gives the <u>MeanFlow identity</u>

$$\boxed{u(z_r,r,t) \;=\; v(z_r,r) \;+\; (t-r)\,\frac{d}{dr}u(z_r,r,t),} \tag{C.7}$$

which is Equation (3.10) of the main text. The total derivative along the trajectory expands through the chain rule into partials,

$$\frac{d}{dr}u(z_r,r,t) \;=\; \partial_z u\cdot \dot z_r + \partial_r u \;=\; v(z_r,r)\,\partial_z u + \partial_r u, \tag{C.8}$$

i.e. exactly the directional (Jacobian–vector) derivative of u along the tangent $(\dot z_r, \dot r, \dot t) = (v,1,0)$.

doi:10.6342/NTU202601285

Why this is trainable. Equation (C.7) relates the field we want (u) to the field we can sample without bias ($v = x_1 - x_0$ along each conditional path, by §C.3) using only quantities available at a single point of a single trajectory — the path integral is gone. A network $u_\theta(z_r, r, t, c)$ is regressed onto the right-hand side of (C.7), with the derivative term (C.8) computed by one forward-mode Jacobian–vector product through u_θ along the tangent $(v, 1, 0)$ and the whole target held under stop-gradient. No higher-order gradients, no teacher, no integration, and no discretisation curriculum are required. A network achieving zero loss satisfies (C.7) everywhere, and therefore reproduces the average velocity of (C.5).

Boundary condition and the FlowCast special case. At $t = r$ the interval factor $(t-r)$ vanishes and (C.7) collapses to $u(z_r, r, r) = v(z_r, r)$: average velocity over a zero-length interval is instantaneous velocity. This is exactly the I-CFM/FlowCast regression of §C.4. MeanFlow therefore generalises the FlowCast baseline — the collapsed ($t = r$) subset of the batch trains the boundary that the $r < t$ subset propagates outward into the interval — which is what makes the comparison of Chapter 4 an A/B test on the objective alone.

Convention note. Geng et al. [7] place data at $s = 0$ and differentiate with respect to the horizon time t, obtaining $u = v - (t - r)\,\frac{d}{dt}u$. Here data sits at $s = 1$, so the differentiation runs through the lower limit of the integral and the sign of the interval term flips to a plus, as in (C.7). The two are the same identity in mirrored time; the loss and sampler of Section 3.6 are internally consistent with the convention used here.

doi:10.6342/NTU202601285